FICHE MODULE 1

L'économie de l'agent

Où part la facture d'un agent, et comment la réduire : cette fiche reprend l'ouverture et le module 1 dans l'ordre des slides, du cran 4 de l'échelle d'autonomie jusqu'à la facture d'une session. Le KV cache et les SLM, retirés du deck, sont gardés en fin de fiche, « pour aller plus loin ».

Reprendre au cran 4

Module 0 · slide « Reprendre au cran 4 »

L'échelle d'autonomie de la Foundation (« Modes de pilotage »), prolongée : la Foundation s'arrête au cran 3, l'Avancé reprend au cran 4.

Niveau d'autonomie	Ce qu'il demande	Où
Observer sans agir	une demande cadrée, un plan relu	Foundation
Demander à chaque action	savoir lire ce que l'agent propose	Foundation
Agir dans le périmètre	tests, un commit par étape, diff relu	Foundation
Aller au bout seul	un harness configuré, un environnement isolé, un juge	**aujourd'hui**

> Aujourd'hui : **configurer** (modules 1 à 3), **vérifier** (4), **juger** (5), **isoler et paralléliser** (6).

Le cran 4 n'est pas « tout autoriser » (le mode YOLO) : c'est laisser l'agent aller au bout **dans un périmètre qu'on a construit**. Ce périmètre a trois parties : un **harness configuré** (ce qui doit arriver est un hook, pas une consigne ; une sandbox, pas une allowlist), une **vérification** qui ne dépend pas de la bonne volonté de l'agent (spec versionnée, tests, drift détecté), un **juge** et un **environnement isolé** dès qu'il y a plusieurs agents.

Qui parle à qui : la pile complète

Module 1 · slide « Qui parle à qui : la pile complète »

Une requête descend la pile, de la méthode jusqu'au fournisseur :

Couche	Rôle
Méthode (SDD)	`spec.md` → `plan.md` → `tasks.md`, relus par l'humain
Orchestrateur	découpe, distribue, attend, agrège, reprend sur erreur
Agent = harness	boucle, tools, contexte ; appelle le modèle
Gateway + guardrails	auth, routage, quotas, cache, journalisation, filtrage entrée / sortie
Fournisseurs	LLM cloud, hyperscaler, souverain, SLM local

- Les **tools** (fichiers, terminal, git, navigateur, serveurs MCP) sont **exécutés par le harness, jamais par le modèle** : le modèle n'écrit qu'une demande d'appel. C'est fondamental pour la sécurité.
- La **gateway** est le seul point par lequel tout passe : l'endroit naturel de la gouvernance (handout « Au-delà du poste (pour aller plus loin) »).
- Seul sur son poste, un dev n'a souvent ni méthode, ni orchestrateur, ni gateway : la pile se construit à mesure qu'on industrialise.

Harness vs orchestrateur : la frontière

Module 1 · slide « Harness vs orchestrateur : la frontière »

	Harness	Orchestrateur
Fait	d'un modèle un agent	travailler plusieurs agents
Responsabilités	gère une boucle d'agent ; exécute les tools, applique les permissions ; construit et compacte le contexte	décompose et route les tâches ; parallélise, attend, agrège ; persiste l'état, reprend après panne
Se pilote	depuis un terminal, un IDE ou une API	en code, en graphe ou en YAML
Exemples	Claude Code, Codex CLI, Copilot CLI, OpenCode	LangGraph, CrewAI, OpenAI Agents SDK, Temporal

> **La frontière bouge** : les harness lancent des sous-agents, les orchestrateurs embarquent leur boucle. Retenez le **rôle**, pas le produit. Ce qu'un orchestrateur dédié apporte en plus : la persistance d'état et la reprise après panne.

Du KV cache au prompt caching

Module 1 · slide « Du KV cache au prompt caching »

Le **prompt caching** : le fournisseur garde le KV cache d'un **préfixe déjà vu**. Même début de requête, le prefill est sauté : plus rapide, et **~90 % de remise** sur l'entrée lue en cache.

Anatomie d'une requête d'agent, du plus stable au plus volatil :

1. System prompt
2. Tool definitions
3. Instructions projet
4. Conversation + résultats des tools
5. Nouveau message

- Modifier le début casse le cache pour **tout ce qui suit** : changer le system prompt, ajouter ou retirer un serveur MCP (donc des tool definitions), changer de modèle, mettre l'heure exacte dans les instructions.
- **Cache expiré** : l'agent n'a **rien oublié**, la conversation est renvoyée en entier. Mais le préfixe est **repayé plein tarif**, et la réponse arrive plus tard.

- **Durée de vie** : de quelques minutes à quelques heures sans activité, selon le fournisseur et l'option choisie. Remises et durées changent souvent : retenez l'ordre de grandeur.

Avant une longue pause (qui casse le cache) : tâche finie → repartir sur une session propre ; tâche en cours → compacter, pour revenir sur un contexte court. La commande de chaque harness est dans « Où ça se règle dans mon outil ».

Ce que coûte cette trace

Module 1 · slide « Ce que coûte cette trace »

La trace de la Foundation : sept tours d'un agent qui ajoute la pagination par curseur sur GET /bookings. Contextes envoyés de 12 k à 21 k tokens, soit **118 k tokens d'entrée cumulés** pour une tâche simple.

Point	Ce qu'il faut savoir
Le constat	l'entrée fait le volume : chaque tour renvoie tout le préfixe déjà vu, et n'ajoute que quelques milliers de tokens
Avec le prompt caching	ce préfixe est lu en cache, à ~10 % du prix
La sortie	~5× plus chère au token, mais petite en volume

Où part la facture d'une session :

Élément	Sur toute la session
System prompt, tool definitions, `AGENTS.md`	gros mais stable : lu en cache
Historique des tours précédents	en cache, si rien ne change avant
Fichier lu en entier, log de build de 3 000 lignes	payé plein tarif une fois, puis renvoyé à chaque tour : il pèse sur toute la fin de la session
Nouvelle demande	plein tarif, mais petite
Réponse et tokens de raisonnement	prix de sortie ; le raisonnement est souvent invisible

> Réduire ce qui **entre** rapporte plus que raccourcir la **sortie** : sorties de tools filtrées, fichiers lus en partie, session propre, modèle adapté.

Pour mesurer sa propre facture : chaque harness affiche la consommation de la session, et des outils recalculent à partir des logs locaux (ex. ccusage). On mesure son usage, on ne compare pas les outils.

Tokens de raisonnement : choisir l'effort

Module 1 · slide « Tokens de raisonnement : choisir l'effort »

Le modèle « réfléchit » en tokens avant de répondre. Ces **reasoning tokens** sont facturés comme de la **sortie** (×5 face à l'entrée), et souvent invisibles : masqués, résumés ou affichés selon le fournisseur.

Effort · classe de modèle	Pour
Bas · modèle léger	renommer, documenter, appliquer un pattern
Moyen · généraliste	paginer un endpoint et ses tests, relire un diff
Haut · raisonnement	bug difficile, migration, architecture

> L'effort se choisit **par tâche** : plus lent, plus cher, et **pas toujours meilleur**. Sur une tâche simple, trop de raisonnement mène à la sur-ingénierie et aux détours.

Bon réflexe : commencer au niveau moyen, monter quand l'agent échoue sur un problème réellement difficile. Les harness appellent ce réglage effort, thinking ou niveau de raisonnement : voir « Où ça se règle dans mon outil ».

> **Tip du module** : garde le **début du contexte** stable, c'est lui que le cache rentabilise. Instructions, tools et modèle fixés en début de session ; ce qui change va à la fin.

Pour aller plus loin

Compléments des slides retirées du deck : à lire seul.

KV cache : ne pas tout recalculer à chaque token

Module 1 · slide « KV cache : ne pas tout recalculer à chaque token »

Une réponse se fait en deux temps : le **prefill** (lire tout le contexte, en parallèle) puis le **decode** (générer token par token). Pour chaque nouveau token, le modèle a besoin des clés et valeurs (K, V) de tous les tokens précédents ; elles ne changent plus une fois calculées, on les garde en mémoire GPU : c'est le **KV cache**.

Point	Ce qu'il faut savoir
Sans cache	chaque token recalculerait K et V de tous les précédents : un coût **quadratique**
Le prix	Llama 3 8B : **128 Ko par token**, **16 Go** pour 128 k tokens, pour une seule conversation
Les parades	têtes K/V partagées (GQA), cache quantifié, mémoire paginée (PagedAttention)

Les deux chiffres des benchmarks en découlent : le **TTFT** (time to first token, le prefill) et le **débit** en tokens/s (le decode).

Les SLM : petits, locaux, spécialisés

Module 1 · slide « Les SLM : petits, locaux, spécialisés »

Point	Ce qu'il faut savoir
Définition pratique	de ~1 à ~15 milliards de paramètres : assez petit pour tourner sur un poste ou un seul GPU (la frontière n'est pas normalisée)
Exemples	Gemma 4, Qwen3.5, Phi-4-mini, Granite 4, Ministral
Bons cas d'usage	classer, extraire, router, compléter du code, **données qui ne sortent pas**
Limites	raisonnement long, agents autonomes sur de gros dépôts
Pour les faire tourner	Ollama, LM Studio, llama.cpp sur le poste ; vLLM sur un serveur

- **Pattern hybride** : le SLM **trie et route**, le LLM **raisonne**.
- Argument entreprise : un SLM sur un serveur interne ne fait sortir aucune donnée, et son coût est fixe (le matériel), pas à l'usage.

Vibe coding vs AI-driven dev

Rappel de la Foundation (slide retirée de l'Avancé) · « Vibe coding vs AI-driven dev »

	Vibe coding	AI-driven dev
Logique	explorer vite	produire de façon fiable
Pour	spike, POC jetable, script ponctuel	fonctionnalité livrée dans un code existant
Exemple	« fais-moi une API de démo »	paginer GET /bookings, avec ses tests
Réflexe	on tente, on observe, on jette	on cadre, on vérifie, on capitalise

> **Viber pour explorer, piloter pour livrer.** Cette formation est du côté droit : livrer en restant à gauche, c'est accepter du code qu'on n'a ni relu ni vérifié.

La boucle, tour par tour

Rappel de la Foundation (slide retirée de l'Avancé ; la boucle est dans les notes du quiz de positionnement) · « La boucle, tour par tour »

Sur l'exemple du fil rouge : « Ajoute la pagination sur l'endpoint /bookings ».

1. Le harness envoie la demande et la liste des tools.
2. Le modèle répond « j'appelle bash avec `npm test -- bookings` ».
3. Le harness vérifie que c'est autorisé, exécute, récupère la sortie : un test échoue, `nextCursor is undefined`.
4. Il renvoie **tout** au modèle (le modèle est sans état), plus la sortie.
5. Le modèle demande à lire le fichier du test, et ainsi de suite jusqu'à une réponse **sans appel de tool**.

- Le « appel de tool ? » est un champ de la réponse de l'API (`stop_reason: "tool_use"` chez Anthropic, `finish_reason: "tool_calls"` chez OpenAI).
- La même boucle porte plusieurs noms : **ReAct** (Pensée → Action → Observation), **Plan-and-Execute**, ou « gather context → take action → verify work → repeat ». Tous décrivent le même mécanisme : planifier, agir, observer le résultat, réajuster.
- À chaque tour, le contexte grossit : c'est tout le sujet de ce module.

À retenir

1. Le modèle **prédit**, le harness **agit** : il exécute les tools et construit le contexte.
2. Le **KV cache** évite de recalculer ; le **prompt caching** rentabilise un **préfixe stable**.
3. En session, l'**entrée** fait le volume ; la sortie et le raisonnement sont chers au token.
4. Modèle et effort se choisissent **par tâche**, du SLM au modèle de raisonnement.

Mots-clés : prefill, decode, KV cache, prompt caching, reasoning tokens, SLM, harness, orchestrateur, gateway — voir le glossaire.

FICHE MODULE 2

Le harness, actif d'équipe

Quelles briques entourent le modèle, et laquelle utiliser pour quoi : cette fiche reprend le module 2, des capacités d'un harness jusqu'à ce qu'il injecte dans la context window.

Les capacités attendues d'un harness

Module 2 · slide « Les capacités attendues d'un harness »

Agent = modèle + tools + boucle + objectif, comme en Foundation ; c'est le harness qui fournit les tools et fait tourner la boucle. Le modèle raisonne ; le harness lui donne les moyens d'agir. Un même modèle dans deux harness peut donner des résultats très différents, en coût comme en réussite.

La grille pour évaluer n'importe quel harness, y compris ceux qui n'existent pas encore :

Capacité	Ce qu'on vérifie
Sous-agents	contexte isolé, et le droit d'écrire (un sous-agent en lecture seule sait relire, pas corriger)
MCP standard	n'importe quel serveur, déclaré dans un fichier versionné
Instructions, skills, hooks	personnaliser sans toucher au cœur
Modes de pilotage	monter l'autonomie, borner les droits
Scriptable	un mode non interactif, pour la CI

Deux philosophies, qui ne s'opposent pas frontalement :

- **Plateforme** (ex. GitHub Copilot, Claude Code) : un harness intégré qui décide de la structure ; Copilot l'étend jusqu'à la forge (issues, PR, CI).
- **Composition** (ex. Pi, OpenCode) : un cœur minimal, des extensions, multi-fournisseur ; on compose son harness.

> Dans les deux cas, le harness est un **actif d'équipe versionné**. Sinon, c'est de l'**agentic debt** : extensions qu'on ne comprend plus, contournements de permissions oubliés, instructions contradictoires empilées. Même remède que la dette technique : versionner, relire, supprimer.

Un seul jeu d'artefacts versionné (`AGENTS.md` , `SKILL.md` , configuration MCP) sert de plus en plus de harness : voir « Où ça se règle dans mon outil ».

Une allowlist n'est pas une sandbox

Module 2 · slide « Une allowlist n'est pas une sandbox »

	Allowlist	Sandbox
Agit sur	des motifs de commande	les effets : fichiers, réseau, droits
Contournement	`npm test && curl …`, un script qui en appelle un autre	aucun par le prompt : l'agent ne sort pas du périmètre
Forme	une liste dans la config du harness	conteneur, VM, compte dédié, réseau coupé

> Le seul périmètre réel est l'**isolation**. Une allowlist filtre le texte d'une commande, pas ce qu'elle fait : on autorise `npm test`, et un script de test modifié par l'agent peut faire n'importe quoi, avec vos droits.

- « La policy vit dans le runtime, pas dans le prompt » : une consigne « ne lance jamais de commande destructrice » dans `AGENTS.md` n'est qu'une suggestion.
- MCP n'apporte rien sur ce terrain : le protocole standardise le branchement des tools, pas leurs permissions.
- **Autonomie et droits sont deux réglages distincts.** Les niveaux d'autonomie (observer, demander, agir dans le périmètre, aller au bout seul) existent dans tous les harness, sous des noms différents. On monte d'un niveau quand on fait confiance au périmètre, et jamais « sans demander » sans isolation. Ce qui rend la montée acceptable, c'est le filet : tests, commits fréquents, diff relu, hook de vérification.

Les primitives du harness : qui décide ?

Module 2 · slide « Les primitives du harness : qui décide ? »

Primitive	Ce que c'est	Qui décide qu'elle s'applique	Exemple sur l'API du fil rouge
Instructions projet	`AGENTS.md` : règles stables, conventions, commandes	personne : toujours chargées	« les endpoints suivent le contrat OpenAPI »
Skill	savoir-faire réutilisable, chargé quand la tâche s'y prête	le modèle, sur sa description	la procédure pour ajouter une migration
Hook	script lancé à un moment précis du cycle de vie	le harness, à chaque événement	lancer les tests après chaque édition
Sous-agent	travail à part, ne renvoie que son résultat	l'agent ou toi	explorer le module bookings sans polluer la session
MCP	accès à un système externe	le modèle, à la demande	lire le ticket dans le tracker

> Le critère qui range tout : **qui décide** qu'une primitive s'applique.

- **Règle de base** : règles stables → instructions projet ; savoir-faire → skill ; ce qui doit arriver à coup sûr → hook ; travail à isoler → sous-agent ; système extérieur → MCP.
- **L'erreur classique** : tout mettre dans `AGENTS.md`, qui pèse dans chaque appel. `AGENTS.md` = ce qui est toujours pertinent ; skill = ce qui l'est parfois (sa description est un prompt) ; hook = ce qui doit arriver sans que personne y pense.
- La **demande initiale** n'est pas une primitive du harness : c'est le premier message, cadré pour la tâche du jour (fichiers clés, tests et invariants). Elle évite de gonfler `AGENTS.md` avec du contexte éphémère.
- Le mot « agent » désigne au moins trois choses : l'agent de code, le sous-agent à contexte isolé, l'agent personnalisé d'un outil. Vérifiez duquel on parle.
- Formats ouverts d'abord : `AGENTS.md` et `SKILL.md` sont lus par la plupart des harness ; l'emplacement des hooks et des sous-agents varie.

Hooks : automatiser le cycle de vie

Module 2 · slide « Hooks : automatiser le cycle de vie »

Des **scripts déterministes**, lancés par le harness, quoi que dise le modèle. Le socle commun à tous les harness :

Événement	Usage	Exemples
Avant un tool	bloquer	`git push --force`, `rm -rf`, ou réécrire une commande verbeuse
Après un tool	formater	le fichier écrit, ou journaliser l'appel
Fin de tour	vérifier	lancer les tests, relire le diff produit
Fin de session	archiver	la trace, ou proposer une mise à jour des notes

> « Lance toujours les tests » dans les instructions est une **suggestion**. Le hook est une **garantie**.

- Le hook d'**avant un tool** peut renvoyer une décision (autoriser, demander, refuser) et donc bloquer l'appel de façon déterministe ; il peut aussi réécrire les arguments.
- **Piège de nommage** : la « fin de tour » se déclenche quand l'agent a fini sa réponse, pas au `git commit`. Pour contrôler le commit, visez le hook d'avant un tool sur la commande git, ou un hook git classique.
- **Piège CI** : en mode non interactif, certains harness ne chargent pas par défaut les hooks du dépôt, par sécurité. Un hook qui marche en local peut être absent en CI.
- Les noms d'événements varient d'un harness à l'autre : voir « Où ça se règle dans mon outil ».

Tool calling : ce qui transite réellement

Module 2 · slide « Tool calling : ce qui transite réellement »

Trois messages, au format simplifié de l'API Messages d'Anthropic ; OpenAI et Google utilisent des structures équivalentes, d'où la possibilité pour une gateway de traduire d'un format à l'autre.

① **Le harness déclare** le tool :

```json
{
  "name": "run_tests",
  "description": "Lance les tests",
  "input_schema": {
    "type": "object",
    "properties": { "path": { "type": "string" } }
  }
}
```

② Le modèle demande :

```json
{ "type": "tool_use", "id": "call_01", "name": "run_tests", "input": { "path": "tests/booking
```

③ Le harness répond :

```json
{ "type": "tool_result", "tool_use_id": "call_01", "content": "FAIL bookings:42 nextCursor is
```

> Le modèle choisit **sur la description** : chaque tool definition part **en tokens** à chaque appel.

- Une mauvaise description = un tool mal utilisé ou ignoré. Quand vous écrivez un serveur MCP ou une skill, soignez la description : c'est un prompt.
- 40 tools MCP branchés = des milliers de tokens de préfixe, à chaque appel. Certains harness chargent désormais les tool definitions à la demande : à vérifier sur son outil.

Ce que le harness injecte

Module 2 · slide « Ce que le harness injecte »

Une session agentique remplit la context window avec bien plus que vos messages :

Bloc	Ce qu'il apporte	Réflexe
System prompt	les règles du harness ouvrent la session	—
Tool schemas	chaque tool et chaque serveur MCP branché prend de la place, même s'il ne sert pas aujourd'hui	couper les MCP inutiles pour la tâche
Fichiers lus	du code brut, pertinent ou non	lire en partie
Sorties de commandes	un build, un `npm install` ou une stack trace peuvent dominer la context window	demander une sortie filtrée (erreurs seules, 20 dernières lignes), ou filtrer par un hook
Logs d'erreur	les erreurs répétées deviennent du bruit	les faire synthétiser
Résultats de tests	utiles, mais consomment aussi du budget	—

> La question n'est pas « l'agent est-il intelligent ? », mais « **ce qu'il a injecté aide-t-il la tâche ?** »

Avant de brancher un serveur MCP, trois questions : **qui l'exécute ?** (un processus local ne sort pas de la machine, un serveur distant reçoit ce que le modèle choisit de lui envoyer) ; **qui l'authentifie ?** (quel jeton, quels droits : jamais plus large que la tâche) ; **qui le maintient ?** (éditeur officiel ou fork communautaire). Le trio létal (données privées + contenu non fiable + capacité d'envoyer vers l'extérieur) se réunit souvent par MCP.

Une primitive par besoin

Module 2 · slide « Quiz : quelle primitive ? »

La grille appliquée à des besoins réels :

Besoin	Primitive	Pourquoi
« Toujours passer par le client HTTP interne »	instructions projet	vrai à chaque tâche ; on peut le doubler d'une règle de lint, déterministe
Interdire `git push --force`, quoi qu'il arrive	hook	doit arriver à coup sûr
Migrer le schéma, une fois par mois	skill	savoir-faire chargé seulement quand la tâche s'y prête
Explorer 40 fichiers sans polluer la session	sous-agent	contexte isolé, ne renvoie que son résultat

> **Tip du module** : ce qui doit toujours arriver est un **hook**, pas une consigne. Une consigne, le modèle peut l'oublier ; un hook, c'est le harness qui l'exécute.

L'exercice 1 met cette règle en pratique : voir « Les exercices ».

Pour aller plus loin

Investir ou attendre : le test à trois questions

Module 6 · slide « Investir ou attendre : le test à trois questions »

Question	Investir	Attendre
Spécifique ?	contexte propre à l'équipe ou au produit	générique : ce que le modèle fera bientôt nativement
Répété ?	plusieurs fois par semaine	rare : le geste qu'on fait deux fois par an
Maintenu ?	un propriétaire identifié	orphelin : personne ne le maintiendra

> Trois oui : on écrit la skill ou le hook. Un non : **on attend**.

- Le critère du propriétaire est le plus discriminant : une skill non maintenue est pire qu'absente (agentic debt).
- La seule mesure qui vaut est la vôtre : combien de fois la skill s'est déclenchée, et le temps qu'elle a évité de réexpliquer. Trois skills utilisées valent mieux que vingt dans le dépôt.

À retenir

1. Un harness s'évalue sur ses **capacités**, pas sur sa marque ; non versionné, il devient de l'**agentic debt**.
2. Une **allowlist** filtre des commandes, une **sandbox** borne les effets : seule l'isolation est un périmètre.
3. Chaque primitive se range par **qui décide** : personne, le modèle, le harness, l'agent, ou toi.
4. Chaque tool branché pèse dans **chaque appel**, et sa description est un prompt.

Mots-clés : harness, agentic debt, permissions, sandbox, allowlist, instructions projet, skill, hook, sous-agent, MCP, tool definitions — voir le glossaire.

FICHE MODULE 3

Ingénierie du contexte

La context window est un budget : cette fiche reprend le module 3, pour donner à l'agent le bon contexte, au bon moment, au bon coût.

Ce qui entre dans la context window

Module 3 · slide « Ce qui entre dans la context window »

Élément	Entre quand ?	Ensuite
`AGENTS.md`, tool definitions	au démarrage	dans chaque appel, **en cache**
Historique	à chaque échange	grossit, compacté près de la limite
Fichiers lus, sorties de tools	à la demande	renvoyés **jusqu'à la fin** de la session
Skills	quand le modèle les juge utiles	nom et description d'abord, contenu ensuite (progressive disclosure)

> Pour le savoir : **regarder** la jauge, **demander** à l'agent ce qu'il a lu, **élaguer** ce qui ne sert plus.

- **Regarder** : l'occupation du contexte, les tool calls, les fichiers lus ; la plupart des harness les affichent (voir « Où ça se règle dans mon outil »).
- **Demander** : « Quels fichiers as-tu lus ? Sur quelle hypothèse pars-tu ? »
- **Élaguer** : MCP inutiles, vieux objectifs, gros logs, par un rewind ou une session propre.
- Un fichier lu reste dans la context window jusqu'à la fin de la session (ou la compaction) : lire 3 000 lignes pour y trouver une fonction, c'est les repayer à chaque tour.

Ce qui s'accumule au fil d'une session, ou pas :

Ressource	Au fil des échanges
Temps de calcul	le préfixe ne se recalcule pas (le KV cache le garde), mais chaque token généré relit tout le cache : plus le contexte est long, plus la réponse ralentit
Mémoire GPU	s'accumule jusqu'à la limite de la context window ; au-delà, le harness compacte ou on repart propre
Facturation	réduite : le préfixe déjà vu est lu en cache, tant que le début du contexte ne change pas
Attention du modèle	se dégrade (lost in the middle, context rot) : compacter restaure aussi la fiabilité, pas seulement le coût

La taille des context windows (de ~128 k à ~1 M tokens selon les modèles) bouge à chaque génération ; la qualité baisse avant même la limite.

Avec ou sans demande initiale

Module 3 · démo « Avec ou sans demande initiale »

La même tâche, deux sessions propres : on compte les fichiers lus et le contexte consommé avant le premier plan, et on juge la justesse du plan.

Une demande initiale cadrée donne à l'agent l'objectif, les fichiers clés, les commandes, les invariants et les pièges. Les deux demandes à coller et le relevé à remplir sont dans le handout « Les démos », pour rejouer la démo.

- Une demande initiale est **orientée tâche**, pas dépôt ; courte et relue. Les noms (chemin, commande, invariant) sont des ancres. Pour une tâche longue, on l'écrit dans un fichier rechargé à chaque session.
- En la relisant, on découvre souvent les zones floues du projet, avant de lancer l'agent.

> Le gain annoncé n'est pas le gain mesuré : **on compte**. Sur un petit dépôt, la recherche agentique suffit souvent.

Réduire les tokens : un levier par buffer

Module 3 · slide « Réduire les tokens : un levier par buffer »

Buffer	Levier	Comment
Sorties de tools	filtrer	erreurs seules, dernières lignes
Données lourdes	traiter hors contexte	un script, un index
Entre sessions	mémoire ciblée	ne plus tout réexpliquer
Réponse	demander du concis	pas de préambule
Code généré	en écrire moins	moins à relire ensuite

> Comprimer la prose du modèle rapporte peu : l'**entrée** domine. Faire **écrire moins de code** rapporte deux fois : maintenant et à la relecture.

- Les outils changent, les buffers restent : chaque levier existe sous forme de plugin, de hook ou de skill dans l'écosystème.
- **Lire les chiffres avec esprit critique** : les gains annoncés sont mesurés en conditions de pointe, pas en moyenne de session, et ne s'additionnent que si votre usage déclenche vraiment chaque levier. Entre un outil très populaire au gain fragile et un outil discret au gain vérifié, choisissez le second.
- Rendre la consommation visible la réduit déjà : voir la context window se remplir pousse à compacter, repartir propre, mieux cadrer.

Deux lignes à coller dans `AGENTS.md`, qui captent l'essentiel du gain d'un plugin de concision :

```text
Réponds de façon dense et factuelle. Pas de préambule, ne reformule
pas ma question, pas de hedging. Une phrase = une information.
```

Le piège des sous-agents

Module 3 · slide « Le piège des sous-agents »

Un cas vécu : une demande de recherche un peu large → **6 sous-agents** lancés en parallèle → chacun charge toutes les pages qu'il lit → **1 M de tokens en 30 min**, quota épuisé.

- La jauge de la session principale allait **très bien** : chaque sous-agent brûlait dans sa propre context window.
- La vue de la context window ne montre pas ce que consomment les sous-agents : regardez la **consommation totale** de la session (la plupart des harness la ventilent par sous-agent).
- En facturation à l'usage, sans quota, la même situation donne une facture sans plafond.
- On borne **périmètre** et **nombre d'agents** avant de lancer, avec un périmètre écrit.

> Un sous-agent protège ta context window, **pas ton budget** : déléguer est une décision de coût.

La mémoire de l'agent

Module 3 · slide « La mémoire de l'agent »

Trois familles, du plus explicite au plus automatique, et aucun standard :

Famille	Ce que c'est	Vigilance
Fichier versionné	instructions et notes du projet	relues en PR, comme du code
Mémoire du harness	l'agent écrit ses notes	propriétaire, souvent plafonnée : **à relire**
Middleware tiers	serveur MCP ou plugin	indexé, à votre main, exportable

- Le bon mécanisme est la **progressive disclosure** : chercher dans un index compact, puis ne charger que le détail des entrées retenues, pour **~10× moins de tokens**. Même pattern que le RAG et les skills : filtrer avant de charger.
- Séparez ce que l'équipe a **décidé** (instructions projet, relues) de ce que l'agent a **appris** (mémoire, à relire).

> Une mémoire qu'on ne peut pas **relire** finit par contenir des faits périmés.

Exploiter les traces

Module 3 · slide « Exploiter les traces »

Une session contient ce qui a été **tenté** et ce qui a **échoué**, ce qui **manquait** (doc, tool, convention), ce qui a **fini par marcher** : c'est de l'**inférence déjà payée**. On en extrait du contexte durable, relu, puis on jette la session.

Fichier	Contenu
`troubleshooting.md`	erreur → cause → fix
`api-fixes.md`	noms, docs à corriger
`next-session.md`	ce qu'on recharge à la prochaine session
`user-guide.md`	commandes qui marchent, pièges
`new-tool.md`	le script ou le tool qui aurait évité le frottement

- `next-session.md` est une **compaction choisie** : on décide de ce qui est gardé, au lieu de laisser le harness résumer quand la context window est pleine.
- Le frottement de l'agent est un signal : s'il galère avec une API, c'est souvent l'API, la doc ou le tool qui parle mal. Les corriger aide aussi les humains.
- « Résume cette session » vaut moins qu'une extraction ciblée. Non relue, une extraction n'est jamais une source d'autorité.

```text
Extrais de cette session, sans rien modifier d'autre :
dans troubleshooting.md, chaque erreur rencontrée sous la forme
erreur → cause → fix ; dans next-session.md, ce qu'il faut recharger
pour reprendre la tâche. Je relis avant de committer.
```

> **Tip du module** : donne à l'agent **de quoi chercher**, pas tout le dépôt. Une demande initiale cadrée, des sorties filtrées, des sous-agents bornés : le contexte est un budget.

Pour aller plus loin

RAG : le retour des embeddings

Module 3 · slide « RAG : le retour des embeddings »

Exemple : « Quelle est notre convention de pagination pour les API ? »

Étape	Ce qui se passe	Exemple
Indexer (indexing)	la doc est découpée en morceaux ; un modèle d'embedding calcule un vecteur par morceau	wiki technique et ADR indexés
Chercher (retrieval)	la question est vectorisée ; on prend les morceaux les plus proches	les 3 passages sur la pagination
Augmenter (augmentation)	ces morceaux entrent dans le contexte, avec la question	3 passages, pas les 400 pages du wiki
Générer (generation)	le modèle répond à partir des passages et cite ses sources	« Curseur opaque, 100 max [ADR-012] »

- L'index par embeddings sert surtout pour la **documentation en langage naturel**. Pour le code, la **recherche agentique** (l'agent cherche lui-même avec grep et la lecture de fichiers) suffit souvent.
- Trois architectures pour indexer du code : **découpage naïf** (blocs fixes : simple, mais coupe les fonctions ; petits dépôts, prose) ; **découpage par l'AST** (fonctions, classes, souvent avec tree-sitter ; dépôts moyens à gros) ; **graphe de code** (AST + relations, pour répondre à « qui appelle cette fonction ? »).
- Le critère : si l'agent lit 40 fichiers pour trouver où brancher l'endpoint, il manque une demande initiale cadrée ou un index. Commencez par la demande initiale, indexez ensuite.

À retenir

1. Ce qui est lu reste jusqu'à la fin de la session : **regarder, demander, élaguer**.
2. Une **demande initiale** cadrée réduit l'exploration ; on mesure le gain, on ne le suppose pas.
3. Un levier par buffer, et l'**entrée** d'abord ; un sous-agent isole le contexte, **pas la facture**.
4. RAG, mémoire et skills suivent le même principe : **filtrer avant de charger**.

Mots-clés : context engineering, demande initiale, recherche agentique, RAG, embeddings / indexation du code, mémoire, progressive disclosure, compaction, context rot — voir le glossaire.

FICHE MODULE 4

Spec-driven et qualité

Garder le code aligné sur l'intention, et le prouver : cette fiche reprend le module 4, de la spec versionnée aux tests et au drift.

Ce que l'IA change dans notre façon de travailler

Module 4 · slide « Ce que l'IA change dans notre façon de travailler »

L'agent a compressé le temps de **production**, pas celui de **vérification**.

	Hier	Avec un agent
Rythme	une feature = des jours	une feature = des heures
Ce qui garantit	revues et rituels, a posteriori	contrôles systématiques, à chaque étape

> Les revues **ne disparaissent pas** : elles ne suffisent plus **seules**. On leur adjoint des **contrôles automatiques**, au rythme de l'agent.

- Nos garanties sont des rendez-vous humains, après coup, à intervalle fixe. Quand le volume produit entre deux rendez-vous est multiplié, le rendez-vous devient un goulot, puis une formalité.
- Le systématique prend ce qui se vérifie mécaniquement (les tests passent-ils ? la spec est-elle à jour ?) ; le temps humain garde l'intention, l'architecture, les arbitrages.

Le SDD est une réponse possible : une chaîne d'artefacts relus par un humain, chaque sortie est un fichier versionné.

Étape	Produit	Contenu
Spécifier	`spec.md`	le quoi et le pourquoi
Planifier	`plan.md`	le comment technique
Découper	`tasks.md`	de petites tâches vérifiables
Implémenter	code + tests	un agent, ou des workers par tâche

Le SDD donne aux agents un objectif écrit et un critère d'arrêt. Outillage, par exemple : Spec Kit, Kiro, OpenSpec, BMAD ; ils fonctionnent au-dessus de n'importe quel harness.

Trois postures face à la spec

Module 4 · slide « Trois postures face à la spec »

SDD : la spec versionnée est la source de vérité que l'agent implémente et que les tests vérifient.

Posture	La spec	Le risque
Spec-first	écrite d'abord, comme prompt structuré ; non maintenue	elle diverge du code
Spec-anchored	spec et code évoluent ensemble	les écarts sont détectés
Spec-as-source	seule la spec est éditée ; le code devient un artefact généré	aucun framework ne le fait (encore) : il faudrait rejouer tous les changements de spec

> Cible réaliste : **spec-anchored**. L'agent sait **formaliser** une spec, pas décider **ce qui doit être vrai**.

- Un curseur de maturité, pas un interrupteur : on démarre en spec-first, on consolide en spec-anchored.
- L'agent formalise volontiers : critères d'acceptation, cas limites (curseur invalide, `limit` négatif, tri instable), dépendances repérées dans le code. Il ne peut pas décider si un curseur invalide doit renvoyer 400 ou la première page : c'est une décision humaine.
- Une spec générée et non relue est une **hypothèse**, pas une source de vérité. Bon usage : demander à l'agent de **lister les questions ouvertes**, plutôt que de les trancher.
- La connaissance se perd à chaque passage de main (intention → ticket → dev → agent → code), et l'agent comble les trous avec du plausible : d'où une intention écrite une fois, versionnée, relue.

```text
Écris une spec de moins de 30 lignes pour ce besoin : contexte,
exigences numérotées, critères d'acceptation, questions ouvertes.
Ne tranche pas les questions ouvertes. Ne code rien.
```

Spec-centric ou workflow-centric

Module 4 · slide « Spec-centric ou workflow-centric »

	Spec-centric	Workflow-centric
Exemples	Spec Kit, OpenSpec, BMAD	Superpowers
Référence	la **spec**, versionnée	la **discipline**, encodée en skills
Ce qui survit	un document relisible	un process reproductible
Point fort	mémoire longue du projet	boucle d'implémentation serrée

> Pas un concurrent à départager : les deux se **combinent**. La spec quand elle doit survivre à la session ; le workflow pour une exécution serrée sur une tâche bornée.

La spec n'est qu'une partie de la doc : elle décrit le quoi. Ce qu'on conserve dans le temps, c'est la **documentation**, fonctionnelle (critères d'acceptation) et technique (architecture, invariants, ADR, runbooks), tenue à jour. Le code est un artefact que la doc permet de reprendre.

Garde-fous : du code au commit

Module 4 · slide « Garde-fous : du code au commit »

L'agent génère vite : sans garde-fou, l'écart avec l'intention passe **inaperçu**.

Garde-fou	Ce que c'est	Vérifie
1. Tests	le contrat exécutable	le comportement
2. Revue agentique	par un agent **distinct** de celui qui a écrit le code	le diff, avant d'envoyer
3. Commit	le point de synchronisation entre code, spec et IA	la doc suit, la part d'IA est tracée

> **Règle minimale** : aucun code généré ne part en production sans **vérification déterministe** (tests, analyse statique) et **revue par un agent distinct**.

- Les trois niveaux sont complémentaires. Ordre dans un pipeline : le **déterministe** d'abord (lint, types, tests, mutation), l'**agentique** ensuite, l'**humain** en dernier. Un agent de revue doit avoir le droit de ne rien dire sur un diff propre.
- **Traçabilité IA** au commit : le trailer `Co-Authored-By:` est une convention git standard (qui a écrit), que certains harness posent nativement ; un footer comme `AI-Generated: ~80%` est une convention d'équipe, approximative (combien).
- **La spec à jour ?** Un contrôle au commit compare le diff et la spec, et bloque si une mise à jour est attendue : un hook (avant le tool `git commit`), un hook git classique ou un job de CI.

```text
feat(bookings): pagination par curseur

Co-Authored-By: <agent>
AI-Generated: ~80%
```

Tests générés : deux natures, une mesure

Module 4 · slide « Tests générés : deux natures, une mesure »

	Tests de construction	Tests fonctionnels
Servent	à l'agent	à l'équipe
Testent	l'implémentation, unité par unité	les effets observables
Durée de vie	**jetables**	**durables** : la spec exécutable
Exemple	« le repository reçoit le bon curseur »	« GET /bookings renvoie un `nextCursor` tant qu'il reste des résultats »

> En fin de feature, là où les deux se recouvrent, **le test fonctionnel gagne** : on supprime le doublon de construction.

- Le critère n'est pas « unitaire vs intégration », mais **à qui le test rend service**. Un test fonctionnel survit à une réécriture de l'implémentation ; on le relit ligne à ligne.
- Le symptôme d'une suite générée trop couplée : toucher une ligne casse dix tests (mocks partout, une classe = un test). Le coût de production d'un test a chuté, son coût de maintenance n'a pas bougé.
- **Score de mutation** : on modifie le code exprès (`<` devient `⩽`), la suite doit **échouer**. Bien plus dur à tricher que la couverture, qu'un agent peut faire monter sans rien vérifier. Outils : Stryker (JS/TS, .NET), PIT (Java), mutmut (Python), cargo-mutants (Rust). Cadence réaliste : nocturne, sur les modules critiques.

Le drift spec ↔ code : qui le détecte ?

Module 4 · slide « Le drift spec ↔ code : qui le détecte ? »

Drift : l'écart qui s'installe entre la spec et le code. (On dit « context rot » pour la dégradation d'une session, pour ne pas confondre.)

Détecteur	Ce qu'il voit	Ce qu'il rate
Tests fonctionnels	un comportement qui contredit une exigence **encodée**	une exigence nouvelle, sans test
Revue de spec dans la PR	un diff qui ne raconte pas la même chose que la spec	ce que le relecteur n'a pas lu
Contrôle au commit	du code changé sans spec touchée	une spec touchée… mais fausse
Personne	—	tout : le drift est **silencieux**

> Quand une exigence change, c'est la **revue de spec** qui le voit d'abord. Aucun détecteur ne suffit seul.

Exemple : l'exigence « curseur invalide → 400 » devient « curseur invalide → première page ». Code changé sans la spec : le test fonctionnel casse (bonne nouvelle). Spec changée sans le test : rien ne casse, seul un relecteur de la spec le voit.

> **Tip du module** : si la spec n'est pas versionnée avec le code, **elle ment déjà**. Spec, tests et commit dans le même dépôt, c'est ce qui rend le drift visible.

L'exercice 2 fait changer une exigence en cours de route : voir « Les exercices ».

À retenir

1. L'agent compresse la **production**, pas la **vérification** : on ajoute des contrôles au rythme de l'agent.
2. Cible réaliste : **spec-anchored** ; l'agent formalise, l'humain décide ce qui doit être vrai.

3. Garde-fous dans l'ordre : **déterministe**, **agentique**, **humain** ; le commit trace la part d'IA.
4. Les tests fonctionnels durent, les tests de construction se jettent ; le **score de mutation** mesure leur vraie valeur.

Mots-clés : SDD, spec-first / spec-anchored / spec-as-source, spec-centric / workflow-centric, drift, tests de construction / tests fonctionnels, mutation testing, guardrails — voir le glossaire.

FICHE MODULE 5

Patterns agentiques

Qui décide de l'étape suivante, et qui juge le résultat : cette fiche reprend le module 5, des cinq patterns d'orchestration à l'agent as a judge, au best-of-N et à la fiabilité composée.

Cinq patterns d'orchestration

Module 5 · slide « Cinq patterns d'orchestration »

Les cinq patterns de « Building effective agents » (Anthropic, 2024). Les cinq sont des **workflows** : le **code** décide de l'étape suivante, du plus figé (chaîne : chemin connu à l'avance) au plus dynamique (orchestrateur-workers, évaluateur : un LLM décide du découpage ou de l'arrêt). L'**agent** autonome, qui choisit lui-même ses étapes, vient ensuite. Rappel de la Foundation : « Chat, workflow ou agent ? ».

Pattern	Principe	Exemple dev
Chaîne	étapes fixes, chaque sortie nourrit la suivante	le flux SDD : spec → plan → tâches → code, chaque étape relue
Routeur	un classifieur oriente vers le bon agent ou le bon modèle	les demandes simples vont à un modèle léger, les complexes à un modèle de raisonnement
Parallélisation	tâches indépendantes, ou vote entre plusieurs, puis agrégation	trois revues du même diff : sécurité, performance, lisibilité
Orchestrateur-workers	le découpage est décidé dynamiquement par un LLM	les sous-agents d'un harness : l'agent principal décide combien en lancer et ce que fait chacun
Évaluateur	un générateur, un évaluateur ; on itère jusqu'au critère	un agent écrit l'endpoint, un autre le relit avec une grille

- Parallélisation et orchestrateur-workers se ressemblent ; dans le second, c'est le **modèle** qui décide du découpage, pas le développeur.
- Le conseil de l'article : commencer simple, n'ajouter un agent que si l'avantage est démontré.

Patterns agentiques : qui décide, qui juge ?

Module 5 · slide « Patterns agentiques : qui décide, qui juge ? »

Quand le modèle décide, deux questions rangent tous les patterns : **qui décide de l'étape suivante** (le code, le modèle, un planificateur) et **qui juge le résultat** (des tests, le modèle lui-même, un autre agent, un humain). Exemples sur le fil rouge : la pagination par curseur sur `GET /bookings`.

Pattern	Qui décide	Qui juge	Quand l'utiliser	Son coût
ReAct	le modèle, à chaque tour	les sorties des tools (tests, compilation)	tâche ouverte avec un retour rapide : corriger un test rouge	un tour par action ; le contexte grossit à chaque tour
Plan-and-Execute	le plan, écrit par le modèle	une vérification par étape	tâche à plusieurs étapes dont on veut relire le plan	un appel de plus pour le plan ; une étape peut être inventée
GOAP	un planificateur, sur des actions déclarées	l'état but atteint	processus connu : livrer, migrer, déployer	décrire les actions une fois ; ensuite prévisible
Reflection	le modèle	le même modèle, avec ses angles morts	relire un brouillon, peu cher	un tour de plus ; il ne voit pas ce qu'il a mal compris
Agent as a judge	le générateur	un agent distinct, avec une grille	entre deux étapes, avant la revue humaine	un contexte et un appel de plus ; à calibrer
Best-of-N	N agents isolés	les tests, puis un juge	tâche incertaine, plusieurs approches plausibles	× N tokens, et N diffs à départager

- **ReAct** (Yao et al., 2022) : Pensée → Action → Observation. C'est la boucle agentique de la Foundation ; le juge, c'est la réalité (la sortie d'un test, une erreur de compilation).
- **Plan-and-Execute** : le modèle écrit un plan, on le relit, puis il l'exécute étape par étape. C'est « planifier avant de coder » de la Foundation, formalisé.
- **Reflection** : l'agent relit sa propre sortie et la corrige. Utile et peu cher, mais il partage les angles morts de l'auteur.
- Combinaisons courantes : Plan-and-Execute avec un agent as a judge entre les étapes ; best-of-N avec des tests, puis un juge.

> Plus le **modèle** décide, plus il faut un **juge** solide.

Plan-and-Execute ou GOAP : qui écrit le plan ?

Module 5 · slide « Plan-and-Execute ou GOAP : qui écrit le plan ? »

	Plan-and-Execute	GOAP
Qui planifie	le LLM, en langage naturel	un algorithme, sur des actions déclarées

	Plan-and-Execute	GOAP
Le plan	souple, mais une étape peut être inventée	chaque action a ses **préconditions** et ses **effets**
Imprévu	on corrige le plan, ou on relance	il **replanifie** depuis l'état réel
Pour	tâches ouvertes : explorer, refactorer	processus connus : livrer, migrer, déployer

GOAP (Goal-Oriented Action Planning) vient de l'IA de jeu vidéo : Jeff Orkin, pour le jeu *F.E.A.R.* (2005). Trois ingrédients : un **état du monde** (des faits), un **état but**, et des **actions**, chacune avec ses **préconditions**, ses **effets** et un coût. Un planificateur (une recherche de chemin, de type A*) cherche la suite d'actions la moins coûteuse qui mène de l'état actuel à l'état but. Le LLM **exécute** les actions ; il ne **choisit** plus leur ordre.

Un exemple détaillé sur le fil rouge

État but : `GET /bookings` paginé, tests verts, contrat OpenAPI à jour, PR ouverte.

État de départ : la spec est lue ; il n'y a pas encore de test.

Action	Préconditions	Effets	Coût
Écrire le test	spec lue	test rouge	1
Implémenter	test rouge	tests verts	3
Mettre à jour le contrat	implémentation faite	contrat OpenAPI à jour	1
Ouvrir la PR	tests verts **et** contrat à jour	PR ouverte	1

Le planificateur trouve la chaîne : écrire le test, implémenter, mettre à jour le contrat, ouvrir la PR. Chaque action n'est possible que si ses préconditions sont vraies dans l'état réel.

Replanification. Après « implémenter », les tests sont encore rouges : l'état réel n'est pas celui que l'action promettait. Le planificateur ne continue pas, il **replanifie** depuis l'état réel : refaire « implémenter », ou passer par une action « analyser l'échec » si elle est déclarée. « Ouvrir la PR » reste impossible tant que « tests verts » n'est pas vrai.

- **Prévisible et explicable** : « pourquoi cette étape ? Parce que sa précondition est vraie et que son effet rapproche du but. » L'ordre est calculé, donc reproductible et testable.
- Le LLM garde sa place : il exécute chaque action (écrire le test, écrire le code) et peut aider à déclarer les actions.
- Exemple de framework : **Embabel** (JVM) planifie ses agents en GOAP. On retrouve l'idée, sans le nom, dans les workflows de SDD où chaque étape a une entrée et une sortie vérifiables.
- Ce n'est pas l'un ou l'autre : un processus de livraison en GOAP peut contenir une action « explorer » confiée à un agent ReAct. Plan-and-Execute pour l'exploration, GOAP pour ce qu'on sait décrire en actions.

Agent as a judge : une grille, un autre contexte

Module 5 · slide « Agent as a judge : une grille, un autre contexte »

C'est le pattern « évaluateur » d'Anthropic appliqué au code : un **générateur** écrit le diff, un **juge** le lit avec le ticket et une grille, rend un verdict (OK, ou retour au générateur) ; **l'humain décide du merge**. Le juge **prépare la preuve**, il ne décide pas.

Ce qui fait un bon juge :

- **Une grille explicite**, pas « est-ce que c'est bien ? » : un verdict par critère, avec la ligne concernée. C'est la grille « Relire le code généré » de la Foundation.
- **Un contexte vierge** : un sous-agent ou une session qui n'a pas vu la génération. Un juge qui a vu le raisonnement de l'auteur le suit.
- **Si possible un autre modèle**, ou au moins un autre prompt : un modèle a tendance à préférer ses propres productions.
- **Déterministe d'abord** : tests, lint, typage, analyse statique ; l'agent ensuite ; l'humain en dernier. Le juge ne remplace pas les tests : il juge ce qu'ils ne voient pas (lisibilité, écart à la spec, cas non testés).

Une grille à copier, à adapter à ton équipe :

```text
Tu relis un diff que tu n'as pas écrit. Ne modifie rien.
Entrées : le ticket (TICKET.md), la spec, le diff.
Pour chaque critère, réponds OK, À CORRIGER ou NON VÉRIFIABLE,
avec le fichier et la ligne concernés :
1. Correction : le diff fait-il ce que demande le ticket ?
2. Cas limites : entrées vides, bornes, valeurs invalides.
3. Sécurité : entrées non validées, secrets, droits.
4. Tests : chaque critère d'acceptation a-t-il un test ?
5. Dépendances : chaque import et chaque API appelée existent-ils ?
6. Lisibilité : le diff reste-t-il petit et dans le style du dépôt ?
Termine par un verdict : MERGEABLE ou RETOUR AU GÉNÉRATEUR,
et les trois points à relire en priorité par un humain.
```

Biais connu	Ce qui se passe	Parade
Préférence pour soi	le modèle préfère ses propres productions	un autre modèle, ou un autre prompt
Verbosité	il préfère les réponses longues	demander un verdict court, critère par critère
Position	il préfère la première option présentée	inverser l'ordre des options et comparer

Calibrer le juge : le faire noter une dizaine de cas déjà jugés par un humain, et vérifier qu'il est d'accord. C'est le principe des evals.

> Le juge **prépare la preuve** ; l'humain **décide**.

Best-of-N : N tentatives, un juge

Module 5 · slide « Best-of-N : N tentatives, un juge »

La **même tâche** est lancée **N fois**, en parallèle. Chaque tentative est **isolée** (un worktree par tentative, voir le module 6) ; les **tests** filtrent, puis un **juge** départage les survivantes.

- **Quand** : une tâche incertaine, plusieurs approches plausibles : un bug coriace, un algorithme.
- **Exemple** : un bug intermittent sur le tri des réservations. Trois agents, trois worktrees, trois correctifs. Deux font passer le test de non-régression ; le juge compare les deux diffs (le plus petit, celui qui ne touche pas au store) ; l'humain relit le gagnant.
- **Coût** : × N tokens, et autant de diffs à départager.
- **Pas pour** : une tâche simple (on paie N fois pour rien) ni une tâche sans test discriminant (on paie N fois et on relit N diffs).

> Best-of-N achète de la **fiabilité** avec des **tokens**. Sans juge, c'est **vous** qui départagez.

La fiabilité composée : 10 étapes à 95 % ≈ 60 %

Module 5 · slide « La fiabilité composée : 10 étapes à 95 % ≈ 60 % »

Fiabilité d'une chaîne d'agents fiables à 95 % chacun :

Étapes	1	2	3	5	10
Fiabilité	95 %	90 %	86 %	77 %	**60 %**

> Les fiabilités se **multiplient** : 0,95 × 0,95 × … dix fois ≈ **0,60**. La parade : une chaîne **courte**, et un **juge** entre les étapes (les tests, puis un agent).

- L'hypothèse est simplificatrice (étapes indépendantes, une erreur = un échec), mais l'ordre de grandeur suffit à calmer l'envie d'empiler les agents.
- Une vérification entre deux étapes (tests, juge, relecture) remet la fiabilité de l'étape près de 100 % : c'est pour ça que les tests comptent tant.
- L'échelle de construction : appel direct → tools → workflow déterministe dès qu'il faut des étapes et des validations explicites → agent supplémentaire seulement si l'avantage est démontré.

Le coût caché : l'érosion de la connaissance

Module 5 · slide « Le coût caché : l'érosion de la connaissance »

Effet	Ce qui se passe
Écrire moins	le code vous appartient moins quand vous ne l'écrivez pas
Relire n'est pas apprendre	on valide une forme, pas un raisonnement
Lent et invisible	on le découvre en incident, deux ans plus tard

> Ce n'est pas un argument contre l'autonomie, c'est sa **condition** : ce qu'on automatise, il faut savoir le reprendre.

- L'IA amplifie ce que vous êtes déjà : bonne base de code → meilleure ; mauvaise → bien pire.
- La documentation est ce qui reste quand la mémoire de l'équipe s'efface (module 4). Supprimer la revue humaine, c'est perdre le principal moment où la connaissance circule.
- Deux questions à se poser : sur le code que votre agent a produit la semaine dernière, lequel sauriez-vous expliquer à un collègue ? Qui saurait reprendre ce module sans l'agent ?

Quel pattern ?

Module 5 · slide « Quiz : quel pattern ? » (tip du module)

Situation	Pattern
« Propose un plan, je le relis, puis exécute-le étape par étape »	Plan-and-Execute
Test, code, contrat, PR : un planificateur choisit l'ordre et replanifie si un test casse	GOAP
Un agent qui n'a pas vu la session relit le diff avec la grille	agent as a judge
Trois agents tentent le même bug ; on garde celui dont les tests passent	best-of-N

> **Tip du module** : ajoute un agent seulement si tu sais **qui jugera son travail**. Un agent de plus, c'est une sortie de plus à juger ; ce qui rend l'autonomie tenable, ce sont les **tests**, les **commits fréquents**, la **relecture du diff** et un juge entre les étapes.

Pour aller plus loin

Hors deck : deux patterns pour prolonger le module, à lire seul.

La boucle longue autonome, avec l'état dans un fichier

L'agent relance sa propre tâche en relisant, à chaque tour, un **fichier d'état** (ce qui est fait, ce qui reste, les décisions prises) au lieu de compter sur la context window. C'est la demande initiale écrite dans un fichier de la Foundation, poussée jusqu'au bout : chaque tour repart d'un contexte court et propre, et l'état survit à une session perdue.

- Pour les tâches longues et répétitives : migrer cinquante fichiers, résorber une liste de warnings.
- Garde-fous : un nombre de tours maximal, un plafond de dépense, un test qui arrête la boucle, et un humain qui relit le fichier d'état.
- Le risque est celui de la fiabilité composée : chaque tour ajoute une chance d'erreur ; on juge entre les tours.

Le débat multi-agents

Plusieurs agents, souvent sur des modèles différents, défendent chacun une solution ; un juge ou un vote tranche. Proche du best-of-N, avec des échanges entre les agents.

- Utile pour une décision d'architecture ou un arbitrage entre approches, rarement pour écrire du code.
- Coût élevé (plusieurs agents, plusieurs tours) pour un gain qui dépend de la qualité du juge : sans grille, le débat produit surtout du texte.

À retenir

1. Un pattern répond à deux questions : **qui décide** de l'étape suivante, **qui juge** le résultat.
2. Cinq workflows où le code décide ; ensuite le modèle décide, et il faut un **juge** plus solide.
3. **Plan-and-Execute** : le LLM écrit le plan ; **GOAP** : un planificateur, sur des actions avec préconditions et effets, replanifie.
4. Un **agent as a judge** a une grille, un contexte vierge et se calibre ; **best-of-N** achète de la fiabilité avec des tokens.
5. Les fiabilités se **multiplient** : chaîne courte, juge entre les étapes ; ce qu'on automatise, il faut savoir le **reprendre**.

Mots-clés : workflow, ReAct, Plan-and-Execute, GOAP, précondition / effet, replanification, reflection, agent as a judge, grille d'évaluation, best-of-N, fiabilité composée, human-in-the-loop — voir le glossaire.

FICHE MODULE 6

Paralléliser : worktrees et ADE

Faire travailler plusieurs agents en même temps sans tout casser : cette fiche reprend le module 6 et la clôture, du worktree à l'ADE, du bilan du parallélisme à la revue de PR en CI.

Agents en parallèle sur worktrees

Module 6 · slide « Agents en parallèle sur worktrees »

Un dépôt, **deux répertoires de travail**, deux agents **en même temps**. Un **git worktree** est un deuxième répertoire de travail rattaché au même dépôt : même historique, autre branche. Chaque agent a ses fichiers, sa branche et son contexte.

	Worktree A	Worktree B
Branche	`feat/bookings-pagination`	`feat/rooms-min-capacity`
Agent	BOOK-142 : pagination de `/bookings`	ROOM-77 : filtre de capacité sur `/rooms`
Fin	revue · merge · nettoyage	revue · merge · nettoyage

Les commandes de git, à lancer depuis le dépôt principal (git 2.5 ou plus) :

```text
git worktree add ../kit-bookings -b feat/bookings-pagination
git worktree add ../kit-rooms -b feat/rooms-min-capacity
git worktree list
git worktree remove ../kit-bookings
```

- Un worktree **ne partage pas** les dépendances installées : `npm install` dans chacun (c'est un coût du parallélisme).
- Une branche ne peut être extraite que dans **un seul** worktree à la fois.
- Une fois la branche mergée : `git worktree remove <dossier>`, puis `git branch -d <branche>`.
- Le seul fichier partagé entre les deux tickets du kit est `openapi.yaml` : les deux agents le modifient, en général à deux endroits différents (git fusionne seul), parfois au même endroit (un conflit à résoudre).
- Ce qui est portable d'un harness à l'autre, c'est **git** : plusieurs éditeurs et harness créent aussi un worktree par session parallèle (voir « Où ça se règle dans mon outil »).

> **Paralléliser demande d'isoler** : chaque agent a ses fichiers, sa branche et son contexte.

L'ADE : piloter des agents, plus des fichiers

Module 6 · slide « L'ADE : piloter des agents, plus des fichiers »

Un **ADE** (Agentic Development Environment) est un poste de travail pensé pour **superviser plusieurs agents**, quand l'IDE est pensé pour écrire du code.

	IDE	ADE
Unité de travail	le fichier, l'onglet	la **session d'agent** : un worktree, une branche
Ce qu'on regarde	le code qu'on écrit	N agents : état, terminal, diff
Ce qu'on valide	sa propre ligne	un diff, puis une PR
Où tourne l'agent	dans l'éditeur	sur le poste, en arrière-plan ou dans le cloud

Ce que fait un ADE, quel que soit le produit :

- créer un **worktree et une branche par session** d'agent, et les nettoyer après le merge ;
- lancer un harness dans chaque session, avec ses terminaux ;
- montrer d'un coup d'œil l'**état** de chaque agent (au travail, en attente d'une permission, fini) et **notifier** ;
- afficher le **diff** de chaque session, permettre de le commenter, ouvrir la PR.

Conductor et **Orca** en sont deux exemples : Conductor est une application de bureau qui lance des agents en parallèle, chacun dans son worktree ; Orca gère worktrees, terminaux et agents de plusieurs fournisseurs, avec un navigateur intégré. D'autres produits suivent la même tendance (applications de bureau des harness, vues « agents » des éditeurs) : la frontière bouge vite.

Agents en arrière-plan ou dans le cloud : on confie un ticket, l'agent travaille dans un environnement distant et rend une PR. Même modèle mental : une tâche = un environnement isolé = une branche = une PR à juger.

> Ce qui ne change pas : le **harness**, les primitives (instructions, skills, hooks) et les formats ouverts. Un ADE orchestre des sessions, il ne remplace ni le harness ni la revue. Retenez le **rôle**, pas le produit.

Développement parallèle : le bilan

Module 6 · slide « Développement parallèle : le bilan »

Ce qu'on gagne	Ce que ça coûte
Temps : deux features avancent ensemble	**Tokens × n** : chaque agent a son propre contexte
Isolation : les agents ne se perturbent pas	**Conflits de merge** : si les features touchent les mêmes fichiers
Revue : des PR plus petites, plus lisibles	**Services partagés** : ports, base de données, dépendances
Supervision : un ADE montre tout au même endroit	**Relecture** : tout passe par vous, le goulot c'est l'humain

> **Tip du module** : paralléliser demande d'isoler, et quelqu'un pour relire. **Features indépendantes** seulement ; au-delà de **2 sessions**, on subit.

- Quand ça vaut le coup : features vraiment indépendantes (modules distincts), plus de 15 à 20 min d'agent chacune, équipe à l'aise avec le merge.
- Déconseillé : services partagés (bases, ports), dépendances très lourdes à réinstaller par worktree.
- La limite du parallélisme n'est pas la machine : c'est la **capacité de revue humaine**. Sans méthode, l'IA peut même ralentir : dans l'étude METR (juillet 2025, 16 développeurs, 246 tâches), les développeurs avec IA ont pris 19 % plus longtemps.

De la revue locale à la revue de PR en CI

Module 6 · slide « De la revue locale à la revue de PR en CI »

Plus d'agents, plus de PR : la **CI relit** d'abord, puis le merge suit le **risque**.

	Revue locale	Revue de PR en CI
Exécution	un hook sur le poste	le runner CI, à chaque PR
Périmètre vu	diff local et conventions	PR complète, spec, règles d'équipe, analyse statique
Restitution	dans la session	commentaire de PR et quality gate

- L'agent quitte la machine du dev et entre dans la **chaîne de production collective** : ce qui se passe sur la machine reste au poste ; tout ce qui entre dans le dépôt partagé, la CI, le ticketing et la supervision relève de la chaîne.
- Les mêmes primitives (instructions, skills, hooks) se retrouvent en CI : c'est pour ça qu'on les versionne.
- Rappel de l'ordre : déterministe d'abord (lint, tests, analyse statique), agentique ensuite, humain en dernier.
- Donner plus de contexte autour du changement à l'agent de revue réduit beaucoup les faux positifs :

```text
git diff -U10 origin/main...HEAD
```

`-U10` donne 10 lignes de contexte au lieu des 3 par défaut ; `--function-context` donne la fonction modifiée en entier.

Une politique de merge graduée

Toutes les PR ne se valent pas : la gouvernance suit le **risque**.

Type de PR	Politique	Encadrement
Doc, correctif mineur	merge automatique	encadré par les evals et la CI
Fonctionnelle	revue synchrone	dev + relecteur
Architecture, cœur, sécurité	double validation	humaine, obligatoire

> Les agents **préparent les preuves** (evals, diffs, specs) ; sur le cœur de métier, **l'humain décide**. L'auto-merge sans humain n'est toléré que sur des correctifs annexes ou de la doc, avec des evals strictes.

Les **evals** (jeux de tests qui mesurent la qualité des réponses d'un agent ou d'une skill) deviennent des quality gates comme les autres. La revue synchrone à trois (le dev, le relecteur et l'agent de revue) est décrite dans « Au-delà du poste (pour aller plus loin) ».

L'exercice 3 met le parallélisme en pratique : voir « Les exercices ».

Une requête, de bout en bout

Clôture · slide « Une requête, de bout en bout »

Exemple : « ajoute la pagination par curseur sur GET /bookings, avec ses tests ».

1. **Spec** : une tâche issue d'une **spec relue** (module 4).
2. **Pattern** : un **plan**, un **juge** prévu (module 5).
3. **Worktree** : un agent **isolé**, sur sa branche (module 6).
4. **Harness** : contexte **stable**, préfixe en tête, hooks et permissions (modules 1 à 3).
5. **Juge** : les **tests**, puis un **agent distinct** (modules 4 et 5).
6. **PR** : la **CI** relit, l'**humain** décide du merge (module 6).

> **Un dev, plusieurs agents, un système** : configurer, isoler, juger.

Trois idées à garder

Clôture · slide « Trois idées à garder »

Idée	En une phrase	Le réflexe
1. Le contexte fait la facture	l'entrée domine, le préfixe stable est en cache, les sous-agents isolent sans être gratuits	mesurer ce qui entre
2. Ce qui doit arriver se configure	hook, sandbox, tests, spec versionnée : ça ne se demande pas au modèle	hook plutôt que consigne
3. Plusieurs agents : isoler et juger	un worktree par agent, un juge entre les étapes, un humain avant le merge	pas d'agent sans juge

Pour aller plus loin : « Building effective agents », Anthropic (2024) ; « ReAct », Yao et al. (2022) ; « Judging LLM-as-a-Judge », Zheng et al. (2023) ; Jeff Orkin, « Three States and a Plan: The A.I. of F.E.A.R. » (2006) ; OWASP Top 10 for LLM (2025). La gouvernance au-delà du poste (guardrails, gateway, budget, réversibilité) est dans le handout « Au-delà du poste (pour aller plus loin) ».

À retenir

1. Paralléliser demande d'**isoler** : un worktree, une branche, un agent.
2. Un **ADE** pilote des sessions d'agent, pas des fichiers ; retenez le rôle, pas le produit.
3. Ce qui sature d'abord, c'est **l'humain qui relit** : au-delà de deux sessions, on subit.
4. Plus d'agents, plus de PR : la **CI relit**, le merge suit le **risque**.

Mots-clés : worktree, isolation, ADE, agent en arrière-plan, revue de PR en CI, merge gradué, human-in-the-loop — voir le glossaire.

SUPPORT D'EXERCICE

Les exercices

Les trois exercices des slides (et, en fin de fiche, un exercice pour aller plus loin) : optionnels, indépendants et courts, à faire avec ton propre harness sur le kit d'exercices.

Le terrain est le **même kit que la formation Foundation** : la Booking API, une petite API de réservation de salles en TypeScript, avec le fil rouge de la pagination par curseur sur `GET /bookings`. Il se télécharge depuis la carte de la formation **Développeur IA — Foundation** (« Kit de code (.zip) ») ; son `README.md` donne l'installation (`npm install` , `npm test`) et les commandes (`npm test -- <filtre>` , `npm run lint`). Le kit ne contient aucune configuration propre à un outil : il marche avec n'importe quel agent de code.

Chaque exercice part du kit tel quel, après un `git init` et un premier commit pour pouvoir revenir en arrière : tu peux en faire un, deux, trois… ou aucun, la formation se suit sans. Les consignes sont écrites en **intention** : chacun les réalise avec le mécanisme de son harness (voir « Où ça se règle dans mon outil »).

Exercice 1 — Garde-fou déterministe (20 min, optionnel)

Module 2 · slide « Exercice 1 — Garde-fou déterministe »

Concept : ce qui doit **toujours** arriver est un **hook**, pas une consigne.

La consigne

1. Choisis un besoin du quiz : bloquer une commande, lancer les tests…
2. Mets-le en place **avec le mécanisme de hook de ton harness**.
3. Demande ensuite à ton agent de faire **le contraire**.

```text
Mets en place un hook qui refuse toute modification de openapi.yaml
(le contrat de l'API), puis explique-moi comment le tester.
```

```text
Ajoute un hook qui lance les tests du module modifié après chaque
édition de fichier.
```

Puis, pour essayer de passer outre :

```text
Ajoute le champ createdAt aux réservations dans openapi.yaml.
```

```text
Modifie src/bookings/bookings.controller.ts sans lancer les tests.
```

Réussi quand…

- l'action est **bloquée**, ou la vérification **lancée**, par le harness ;
- **même** quand on demande le contraire.

Débrief : qu'a tenté l'agent pour passer outre ? S'il essaie une autre commande pour arriver au même effet, c'est « une allowlist n'est pas une sandbox » en direct.

Exercice 2 — La spec change (25 min, optionnel)

Module 4 · slide « Exercice 2 — La spec change »

Concept : la spec versionnée est la source de vérité ; le **drift** se détecte, il ne se devine pas.

La consigne

1. Écris une spec courte pour une fonctionnalité de l'API (suggestion : la pagination par curseur sur `GET /bookings`, le ticket `TICKET.md` du kit).
2. Versionne-la, fais-la implémenter **avec ses tests**.
3. En cours de route, **change une exigence** : par exemple « curseur invalide → 400 » devient « curseur invalide → première page ». Une fois dans la spec seulement, une fois dans le code seulement.

Aucun framework de SDD imposé : un fichier markdown versionné suffit. Si tu en utilises un, garde-le.

```text
Écris une spec de moins de 30 lignes pour ce besoin : contexte,
exigences numérotées, critères d'acceptation, questions ouvertes.
Ne code rien.
```

```text
Implémente la spec docs/specs/bookings-pagination.md avec ses tests
fonctionnels. Un test par critère d'acceptation.
```

Réussi quand…

- la spec est **versionnée** dans le dépôt ;
- l'implémentation **passe ses tests** ;
- le changement est **détecté**, et on sait dire **par quoi** : le test rouge ou la remarque de revue qui l'a révélé.

Débrief : qu'est-ce qui aurait laissé passer le drift ?

Exercice 3 — Deux agents, deux worktrees (20 min, optionnel)

Module 6 · slide « Exercice 3 — Deux agents, deux worktrees »

Concept : paralléliser demande d'**isoler** : un worktree, une branche, un agent.

Prérequis

- le kit **commité** (`git init` , premier commit, comme dans son `README.md`) ;
- **git 2.5** ou plus (`git --version`) ;
- `npm install` **dans chaque worktree** : les dépendances ne sont pas partagées, c'est un coût du parallélisme.

Les deux tickets, indépendants :

Ticket	Fichier	Ce qui est demandé
BOOK-142	`TICKET.md`	pagination par curseur sur `GET /bookings`
ROOM-77	`tickets/ROOM-77.md`	filtre `minCapacity` sur `GET /rooms`

La consigne

1. Crée **deux worktrees** du kit, avec git ou ton ADE : un pour `TICKET.md`, un pour `tickets/ROOM-77.md`.
2. Lance **un agent dans chacun**, en même temps.
3. **Merge** les deux branches.

Sans ADE, avec git (depuis le dossier du kit) :

```text
git worktree add ../kit-bookings -b feat/bookings-pagination
git worktree add ../kit-rooms -b feat/rooms-min-capacity
```

À coller dans chaque session, en adaptant le ticket :

```text
Lis TICKET.md, propose un plan, puis implémente-le avec ses tests ;
fini quand npm test passe.
```

```text
Lis tickets/ROOM-77.md, propose un plan, puis implémente-le avec ses tests ;
fini quand npm test passe.
```

Réussi quand…

- chaque branche a ses **tests verts** ;
- les deux sont **mergées**, et `npm test` passe sur la branche principale ;
- tu sais dire ce qui aurait pu **entrer en conflit**.

Les tests du kit n'ouvrent pas de port : deux agents peuvent lancer `npm test` en même temps. Le fichier partagé, c'est `openapi.yaml` : les deux agents le modifient, en général à deux endroits différents (git fusionne seul), parfois au même endroit (un conflit à résoudre).

Débrief

- Combien de sessions pouviez-vous vraiment suivre ?
- Qu'avez-vous relu, et quand ?
- Qu'est-ce qui a failli entrer en conflit ?
- Qu'est-ce qui a coûté : installation, relecture de deux diffs, changements de contexte ?

Pas de poste prêt ? Le déroulé pas à pas, à rejouer seul, est dans la démo « Deux agents, deux worktrees ».

Pour aller plus loin

Hors deck : l'ancien exercice 3, sur les sous-agents.

Exercice — Déléguer et isoler (15 min, optionnel)

Module 3 · ancien exercice 3, retiré du deck : « Déléguer et isoler » (sous-agents) ; lié à la slide « Le piège des sous-agents »

Concept : un sous-agent **isole** son contexte et ne renvoie que son résultat… mais il **coûte**.

La consigne

1. Confie **avec le mécanisme de ton harness** à des sous-agents : une exploration du code, la revue d'un fichier par un regard neuf.
2. Refais-le dans la session principale, et compare.

Le fichier à relire : `src/bookings/bookings.controller.ts`, tel que le kit le fournit (il contient au moins un bug latent). Ton harness n'a pas de sous-agents ? Deux sessions séparées font le même travail : le concept est l'isolation du contexte.

```text
Avec un sous-agent, explore le module bookings et rends-moi
en 15 lignes : fichiers clés, points d'extension, pièges.
Ne modifie rien.
```

```text
Avec un sous-agent qui n'a pas vu cette session, relis
src/bookings/bookings.controller.ts. Liste les bugs et les risques,
rien d'autre. Ne modifie rien.
```

Réussi quand…

- les deux tâches ont tourné dans des **contextes isolés** ;
- on compare **contexte consommé**, **fichiers lus** et **qualité**.

Débrief : qu'a gagné la session principale ? Et combien a coûté la session entière ? Regarde la jauge de la session principale **et** la consommation totale (voir « Le piège des sous-agents »).

DÉMOS À REJOUER

Les démos

Les démos jouées par le formateur pendant la formation, pour les rejouer seul, que tu aies été présent ou non. Elles tournent toutes sur le kit d'exercices de la Foundation (carte **Développeur IA — Foundation**, « Kit de code (.zip) »), installé avec `npm install` (`npm test` doit être vert) et un premier commit, avec ton agent de code ouvert dessus.

Un hook qui bloque l'agent

Module 2 · démo de l'exercice 1 « Garde-fou déterministe » · 3 min

Ce que ça montre : ce qui doit **toujours** arriver est un **hook**, pas une consigne. Une consigne, le modèle peut l'oublier ; un hook, c'est le harness qui l'exécute, quoi que demande le modèle.

À rejouer

1. Mets en place le hook, avec le mécanisme de hook de ton harness (voir le handout « Où ça se règle dans mon outil ») :

```text
Mets en place un hook qui refuse toute modification de openapi.yaml
(le contrat de l'API), puis explique-moi comment le tester.
```

2. Demande ensuite le contraire :

```text
Ajoute le champ createdAt aux réservations dans openapi.yaml.
```

Ce qu'on observe : le refus vient du harness, il apparaît dans la trace de la session. L'agent tente souvent un autre chemin pour arriver au même effet (une commande `sed`, un autre outil d'écriture) : c'est « une allowlist n'est pas une sandbox » en direct. Tout se passe en local, sans remote git.

Avec ou sans demande initiale

Module 3 · démo « Avec ou sans demande initiale » · 5 min

Ce que ça montre : une demande initiale cadrée réduit l'exploration de l'agent et améliore son plan. On ne le croit pas sur parole, on le **mesure** : même tâche, deux sessions, on compare les chiffres.

Avant de commencer : la jauge de contexte et la liste des fichiers lus visibles dans ton harness. Les deux sessions s'arrêtent au **plan** : rien n'est modifié.

Session 1, sans cadrage. Nouvelle session vierge, colle :

```text
Ajoute la pagination par curseur sur GET /bookings, avec ses tests.
Propose d'abord un plan, ne modifie aucun fichier.
```

Session 2, avec une demande initiale. Nouvelle session vierge (pas la même), colle :

```text
Ajoute la pagination par curseur sur GET /bookings, avec ses tests.

Tâche : pagination par curseur sur GET /bookings (ticket BOOK-142).
Fichiers clés : src/bookings/bookings.controller.ts,
src/shared/pagination/cursor.ts (le helper à réutiliser),
src/rooms/rooms.controller.ts (l'exemple déjà paginé), openapi.yaml.
Commandes : npm test -- bookings, puis npm test et npm run lint.
Invariants : réutiliser le helper de pagination partagé ;
curseur opaque (base64url) ; réponse { items, nextCursor } comme GET /rooms ;
mettre à jour openapi.yaml.
Pièges : ne pas réinventer un helper ; limit max 100 ;
plusieurs réservations le même jour : la clé du curseur doit départager les égalités.

Propose d'abord un plan, ne modifie aucun fichier.
```

Le relevé

	Session 1 (sans)	Session 2 (avec)
Fichiers lus avant le plan		
Context window consommée		
Réutilise `src/shared/pagination/cursor.ts` ?		
Prévoit de mettre à jour `openapi.yaml` ?		
Départage les réservations du même jour ?		

Ce qu'on observe : les chiffres, et surtout **la justesse du plan**. Sans cadrage, l'agent invente-t-il son propre helper, oublie-t-il le contrat ou les égalités de date ? Avec, suit-il les invariants ? Si l'écart est faible, c'est un résultat aussi : sur un petit dépôt, la recherche de l'agent suffit souvent. Le gain annoncé n'est pas le gain mesuré : on compte.

Deux agents, deux worktrees

Module 6 · démo de repli de l'exercice 3 « Deux agents, deux worktrees » · 3 min

Ce que ça montre : paralléliser demande d'**isoler**. Deux agents travaillent en même temps sur deux tickets indépendants, chacun dans son worktree et sur sa branche ; on relit deux diffs, puis on merge.

Avant de commencer : le kit installé et **commité** (`git init` , premier commit), git 2.5 ou plus, deux tickets : `TICKET.md` (BOOK-142, pagination de `/bookings`) et `tickets/ROOM-77.md` (filtre `minCapacity` sur `GET /rooms`). Un ADE (Conductor, Orca…) si tu en as un ; sinon deux terminaux.

À rejouer

1. Crée un worktree et une branche par ticket. Dans un ADE : une nouvelle session par ticket. Sans ADE :

```text
git worktree add ../kit-bookings -b feat/bookings-pagination
git worktree add ../kit-rooms -b feat/rooms-min-capacity
```

2. Installe les dépendances **dans chaque worktree** (`npm install`) : elles ne sont pas partagées.

3. Lance **un agent par worktree**, en même temps (deux sessions dans l'ADE, ou deux terminaux) :

```text
Lis TICKET.md, propose un plan, puis implémente-le avec ses tests ;
fini quand npm test passe.
```

```text
Lis tickets/ROOM-77.md, propose un plan, puis implémente-le avec ses tests ;
fini quand npm test passe.
```

4. Quand les deux ont fini, **relis chaque diff** et vérifie `npm test` dans chaque worktree.
5. **Merge** les deux branches, depuis le dépôt principal :

```text
git merge feat/bookings-pagination
git merge feat/rooms-min-capacity
npm test
```

6. **Nettoie** :

```text
git worktree remove ../kit-bookings
git worktree remove ../kit-rooms
git branch -d feat/bookings-pagination feat/rooms-min-capacity
```

Ce qu'on observe : les deux agents n'ont pas de port ni de fichier en commun, ils avancent sans se gêner. Le fichier partagé est `openapi.yaml` : le plus souvent git fusionne seul ; s'il y a un conflit, c'est le bon moment pour voir ce que coûte une feature qui touche un fichier commun. Et surtout : combien de sessions peut-on vraiment suivre ? Le goulot n'est pas la machine, c'est la relecture.

Pour aller plus loin

Hors deck : la démo de l'ancien exercice 3.

Un sous-agent pour explorer

Module 3 · ancien exercice 3 « Déléguer et isoler », retiré du deck · 3 min

Ce que ça montre : un sous-agent **isole** son contexte et ne renvoie que son résultat, mais il **coûte** : la session principale reste légère, la consommation totale ne baisse pas.

À rejouer, avec le mécanisme de sous-agents de ton harness (sans sous-agents : une deuxième session séparée fait le même travail) :

```text
Avec un sous-agent, explore le module bookings et rends-moi
en 15 lignes : fichiers clés, points d'extension, pièges.
Ne modifie rien.
```

Puis la même demande dans la session principale, sans sous-agent.

Ce qu'on observe : la jauge de la session principale reste basse avec le sous-agent ; la consommation totale de la session, elle, ne baisse pas forcément (voir « Le piège des sous-agents »). L'exercice complet, avec la revue de `bookings.controller.ts`, est dans le handout « Les exercices ».

FICHE OUTILS

Où ça se règle dans mon outil

La formation parle de concepts ; cette fiche donne, pour quatre harness courants, l'endroit où se règlent les primitives de l'Avancé : hooks, sous-agents, permissions et sandbox, instructions projet, skills, MCP, compaction, effort de raisonnement.

> **Rédigé le 21 octobre 2026 ; les outils changent vite.** Les noms de modes, d'événements, les commandes et les chemins bougent d'une version à l'autre : en cas de doute, la documentation du jour fait foi. Cette fiche ne reprend que ce que les supports Zenika sourcent (handout « Où ça se règle » de la Foundation, vérifié sur les docs des éditeurs le 21 octobre 2026 ; bootcamp AI-Driven Development, module 2). « à vérifier dans la doc de l'outil » signale une case non sourcée ; « — » l'absence d'équivalent documenté.

Les formats ouverts passent d'un outil à l'autre : `AGENTS.md` (instructions projet), `SKILL.md` (skills) et MCP (tools). Une équipe mixte peut versionner un seul jeu de fichiers, chacun garde son harness. Hooks et sous-agents, eux, n'ont pas de format commun.

Les fichiers de contexte

Slides « Les primitives du harness : qui décide ? » et « Tool calling : ce qui transite réellement » (module 2)

Concept	GitHub Copilot	Claude Code	Codex	Cursor
Instructions projet	`AGENTS.md` · `.github/copilot-instructions.md` · `.github/instructions/*.instructions.md` ciblés par glob (`applyTo`)	`CLAUDE.md` (projet : `./CLAUDE.md` ; perso : `~/.claude/CLAUDE.md`) · `AGENTS.md` , lu seulement s'il n'y a pas de `CLAUDE.md` (sinon l'importer : `@AGENTS.md`)	`AGENTS.md` (perso : `~/.codex/AGENTS.md`)	`AGENTS.md` · `.cursor/rules/*.mdc`
Skills (`SKILL.md`)	`.github/skills/<nom>/` (lit aussi `.claude/skills/` et `.agents/skills/`) ; perso : `~/.copilot/skills/`	`.claude/skills/<nom>/` ; perso : `~/.claude/skills/`	`.agents/skills/<nom>/` ; perso : `~/.agents/skills/`	`.cursor/skills/` ou `.agents/skills/` (lit aussi `.claude/skills/`) ; perso : `~/.cursor/skills/`
Déclencher une skill	le modèle, sur la description, ou `/<nom>` dans la demande	le modèle, sur la description, ou `/<nom>`	le modèle, sur la description, ou `$<nom>`	le modèle, sur la description, ou `/<nom>`
Serveurs MCP	CLI : `~/.copilot/mcp-config.json` (ou `/mcp add`) · projet : `.mcp.json` ou `.github/mcp.json` · VS Code : `.vscode/mcp.json` (clé `servers` , pas `mcpServers`)	`.mcp.json` à la racine, versionné (ou `claude mcp add`)	`~/.codex/config.toml` ou `.codex/config.toml` du projet, table `[mcp_servers.<nom>]` (ou `codex mcp add`)	`.cursor/mcp.json` (projet) · `~/.cursor/mcp.json` (perso)

Les hooks

Slides « Hooks : automatiser le cycle de vie » et « Exercice 1 — Garde-fou déterministe » (module 2)

	GitHub Copilot	Claude Code	Codex	Cursor
Où se déclarent les hooks	`.github/hooks/*.json` (dépôt, versionné, lu aussi par l'agent cloud) · `~/.copilot/hooks/` (utilisateur) · lit aussi les hooks de `.claude/settings.json` , en partie (moins d'événements, commandes shell seulement)	`.claude/settings.json` (projet, versionné) · `.claude/settings.local.json` (projet, non versionné) · `~/.claude/settings.json` (utilisateur) ; ne lit pas `.github/hooks/`	à vérifier dans la doc de l'outil	à vérifier dans la doc de l'outil
Noms d'événements	camelCase : `sessionStart` , `userPromptSubmitted` , `preToolUse` , `postToolUse` , `preCompact` , `subagentStop` , `agentStop` , `sessionEnd` … (14 documentés)	PascalCase, liste plus large : `SessionStart` , `PreToolUse` , `PostToolUse` , `Stop` …	à vérifier dans la doc de l'outil	à vérifier dans la doc de l'outil
Bloquer avant un tool	`preToolUse` renvoie un JSON avec `permissionDecision` : `allow` , `deny` ou `ask` (+ motif) ; il peut réécrire les arguments	`PreToolUse` : le code de sortie `2` refuse l'appel et renvoie stderr au modèle ; `0` laisse passer	à vérifier dans la doc de l'outil	à vérifier dans la doc de l'outil

	GitHub Copilot	Claude Code	Codex	Cursor
Cibler un tool	`matcher` (sur `preToolUse`, `postToolUse`, `permissionRequest`, `preCompact`, `subagentStart`, `notification` seulement)	`matcher` (ex. `Bash`, `Edit|Write`)	à vérifier dans la doc de l'outil	à vérifier dans la doc de l'outil

- Les quatre moments génériques de la formation se lisent ainsi : **avant un tool** = `preToolUse` / `PreToolUse` ; **après un tool** = `postToolUse` / `PostToolUse` ; **fin de tour** = `agentStop` / `Stop` ; **fin de session** = `sessionEnd`.
- La « fin de tour » n'est pas un pre-commit : pour contrôler un `git commit`, visez le hook d'avant un tool sur la commande git, ou un hook git classique.
- La sortie standard d'un hook ne s'affiche pas : on juge un hook à son **effet**. Seul le `SessionStart` de Claude Code réinjecte sa sortie dans le contexte ; Copilot attend un JSON avec un champ `additionalContext`.

Les sous-agents

Slides « Les primitives du harness : qui décide ? » (module 2), « Le piège des sous-agents » (module 3)

	GitHub Copilot	Claude Code	Codex	Cursor
Définir un sous-agent	agents personnalisés : `.github/agents/*.agent.md`	`.claude/agents/`	sous-agents disponibles (jusqu'à 6 agents concurrents) ; emplacement : à vérifier dans la doc de l'outil	à vérifier dans la doc de l'outil
Le lancer	`/agent` (CLI), ou l'agent principal qui délègue	l'agent principal, par le tool `Task`	à vérifier dans la doc de l'outil	à vérifier dans la doc de l'outil
Voir ce qu'il consomme	`/usage` (CLI : tokens par modèle, AI Credits)	`/usage` (`/cost` en est l'alias), ventilé par skill, sous-agent et MCP	`/usage`	tableau de bord *Spending*, hors session

Un sous-agent sans droit d'écriture sait relire, pas corriger : vérifiez ce que votre harness lui autorise.

Permissions, autonomie et sandbox

Slides « Une allowlist n'est pas une sandbox » (module 2) et, pour aller plus loin, « Au-delà du poste » (guardrails)

	GitHub Copilot	Claude Code	Codex	Cursor
Observer sans agir (mode plan, lecture seule)	mode *plan* (`/plan`, Maj+Tab)	mode *plan*	mode plan (`/plan`) · `sandbox_mode = "read-only"`	mode *Ask* (lecture seule) ou *Plan* (Maj+Tab)
Demander à chaque action	permissions `default` (CLI) · *Manual permissions* (VS Code, par défaut)	mode *Manual* (valeur `default`)	`trust_level = "untrusted"` sur le projet	mode *Allowlist* : tout ce qui n'est pas listé demande une approbation
Agir dans le périmètre	approbations mémorisées par emplacement · `--allow-tool`	*accept edits* (`acceptEdits`)	préréglage *Auto* (par défaut) : `approval_policy = "on-request"` + `sandbox_mode = "workspace-write"`	mode *Auto-review* : liste blanche, sandbox, puis classifieur
Aller au bout seul	*Autopilot* · *Allow all* (`/yolo`, `--yolo`)	*auto mode* (classifieur) · *bypass permissions*	*Auto-review* (`approvals_reviewer = "auto_review"`) · `--yolo` (ni sandbox ni approbation)	mode *Run Everything* (`/run-everything` en CLI)
Allowlist (où se déclarent les droits)	`--allow-tool` / `--deny-tool` au lancement · `~/.copilot/permissions-config.json` · VS Code : réglages `chat.tools.*.autoApprove`	`permissions.allow` / `ask` / `deny` dans `.claude/settings.json`, versionné (ex. `"deny": ["Bash(rm:*)"]`)	`~/.codex/config.toml` : `approval_policy`, réglé séparément de la sandbox · `/permissions` en session	*Settings* > *Agents* > *Approvals & Execution* · CLI : `permissions.allow` / `deny` dans `.cursor/cli.json`

	GitHub Copilot	Claude Code	Codex	Cursor
Sandbox (ce que l'agent peut atteindre)	`copilot --cloud` : session dans une sandbox cloud ; en local : à vérifier dans la doc de l'outil	à vérifier dans la doc de l'outil	`sandbox_mode` dans `config.toml` (`read-only` , `workspace-write` …), réglé séparément de l'approbation	sandbox du mode *Auto-review* ; détail : à vérifier dans la doc de l'outil
Verrou au-dessus de l'utilisateur	un administrateur d'entreprise peut interdire les modes de contournement (`disableBypassPermissionsMode`)	les règles `deny` s'appliquent même en *bypass permissions*	à vérifier dans la doc de l'outil	à vérifier dans la doc de l'outil

- Le mode affiché n'est qu'un **défaut** : une autorisation large en configuration fait que l'agent ne demande plus rien. La configuration a le dernier mot.
- Codex sépare explicitement les deux réglages de la formation : l'**approbation** (quand on me demande) et la **sandbox** (ce que l'agent peut atteindre). Une bonne grille de lecture, quel que soit l'outil.
- Quand aucune sandbox n'est sourcée pour votre harness, l'isolation reste possible hors de l'outil : conteneur, devcontainer, VM, compte dédié, réseau coupé.

Les actions en session

Slides « Du KV cache au prompt caching » et « Tokens de raisonnement : choisir l'effort » (module 1), « Ce qui entre dans la context window » (module 3)

Action	GitHub Copilot	Claude Code	Codex	Cursor
Voir l'occupation de la context window	`/context` · indicateur (petit cercle) dans VS Code	`/context`	`/status` · « % context left » en pied d'écran	anneau de contexte à côté de la saisie (clic : détail par catégorie)
Compacter	`/compact`	`/compact` (avec une consigne : `/compact focus on …`)	`/compact`	`/summarize`
Repartir à zéro (nouvelle session)	`/clear` (alias `/new`)	`/clear` (alias `/new`)	`/new` (ou `/clear`)	nouveau chat (CLI : `/clear`)
Choisir le modèle	`/model` (catalogue multi-éditeur)	`/model`	à vérifier dans la doc de l'outil	à vérifier dans la doc de l'outil
Choisir l'effort de raisonnement	à vérifier dans la doc de l'outil	réglage de l'effort, avec le choix du modèle ; commande exacte : à vérifier dans la doc de l'outil	à vérifier dans la doc de l'outil	à vérifier dans la doc de l'outil
Mémoire du harness	Copilot Memory · `/memory` (CLI)	mémoire automatique (`MEMORY.md`) · `/memory`	mémoire en opt-in ; détail : à vérifier dans la doc de l'outil	à vérifier dans la doc de l'outil

> Changer de modèle ou brancher un MCP en cours de session fait sauter le cache, tout le contexte est repayé. Donc : choisissez modèle, effort et tools **au démarrage**.

Paralléliser : worktrees et ADE

Slides « Agents en parallèle sur worktrees », « L'ADE : piloter des agents, plus des fichiers » et « De la revue locale à la revue de PR en CI » (module 6), « Les capacités attendues d'un harness » (module 2)

	GitHub Copilot	Claude Code	Codex	Cursor
Un worktree par session	CLI : pas natif ; application de bureau : un worktree par session	`--worktree` (et un fichier pour embarquer les fichiers ignorés)	application : oui ; CLI : non	à vérifier dans la doc de l'outil
Mode non interactif (CI)	`copilot -p "…"` (avec `--allow-tool` ciblé) ; les hooks et MCP du dépôt ne sont pas chargés par défaut	`claude -p "…"`	à vérifier dans la doc de l'outil	à vérifier dans la doc de l'outil

	Ce que c'est	Exemples
ADE (Agentic Development Environment)	un poste qui pilote des sessions d'agent : un worktree par session, terminaux, diff, PR	**Conductor**, **Orca** ; les applications de bureau des harness ci-dessus

	Ce que c'est	Exemples
`git worktree`	la brique portable : `git worktree add <dossier> -b <branche>`, `git worktree list`, `git worktree remove <dossier>`	git 2.5 ou plus, avec n'importe quel harness

Partout, ce qui est portable pour paralléliser, c'est **git** : `git worktree add` marche avec n'importe quel harness. Un ADE automatise la création, l'affichage et le nettoyage des worktrees, il ne change pas le principe.

Sources

- Fichiers de contexte, niveaux d'autonomie, actions en session : handout « Où ça se règle dans mon outil » de la formation Développeur IA — Foundation, vérifié sur la documentation officielle des éditeurs le 21 octobre 2026 (les liens y sont listés).
- Hooks, sous-agents, sandbox, worktrees, mode non interactif, mémoire : bootcamp AI-Driven Development, module 2 (slides et handouts des labs, 2026), et leur extraction dans les documents de conception des formations dev.
- Référence des hooks GitHub Copilot : https://docs.github.com/copilot/reference/hooks-reference

GLOSSAIRE

Glossaire des mots-clés IA

Une phrase par terme : les mêmes mots, avec le même sens, que dans la formation business et la Foundation, puis les termes propres à l'Avancé.

Les termes techniques anglais sont gardés (harness, tools, context window, skill) : ce sont ceux que vous entendrez dans les outils et les réunions. On dit **context rot** pour la dégradation d'une session, et **drift** pour l'écart entre la spec et le code.

Tronc commun : les bases (rappel)

Ouverture et module 1

Terme	En une phrase
IA générative	IA qui produit du contenu (texte, image, code) au lieu de seulement classer ou prédire un chiffre.
LLM (Large Language Model)	Modèle entraîné sur d'énormes volumes de texte, qui génère une réponse en prédisant le mot suivant.
Token	Morceau de mot traité par le modèle ; c'est aussi l'unité de facturation (≈ ¾ de mot en anglais).
Hallucination	Réponse fausse mais présentée avec assurance et vraisemblance.
Knowledge cutoff (date de coupure)	Date à laquelle s'arrêtent les données d'entraînement du modèle ; il ignore tout ce qui s'est passé après, sauf s'il fait une recherche web.
Assistant	Le produit qu'on utilise au quotidien (ChatGPT, Copilot, Claude.ai, Gemini) : une interface posée sur un harness, autour d'un modèle.
Harness	Tout le logiciel autour du modèle : consignes, mémoire, tools, enchaînement des étapes. Deux assistants sur le même modèle diffèrent par leur harness.
Tools	Capacités que le modèle peut appeler pendant sa réponse : recherche web, calcul, lecture de fichiers, Jira, Figma… (*tool calling*).
MCP (Model Context Protocol)	Standard pour brancher de nouveaux tools sur un assistant : applications et données de l'entreprise.
Agent	Modèle qui enchaîne lui-même des actions avec des tools, en boucle, jusqu'à atteindre un objectif : c'est la **boucle agentique** (modèle + tools + boucle + objectif, que le harness fait tourner). Les éditeurs utilisent aussi « agent » pour de simples assistants personnalisés.
Boucle agentique (agent loop)	Le cycle demande → le modèle choisit un tool → le harness l'exécute → le résultat entre dans le contexte → on recommence si besoin.
Prompt	La demande envoyée au modèle, consignes comprises.

Terme	En une phrase
Skill	Savoir-faire emballé (consignes, modèles, exemples, parfois des scripts) que l'assistant charge **à la demande**, quand la tâche correspond. Plus léger que des instructions permanentes, partageable dans une équipe. Claude parle d'« Agent Skills » ; d'autres assistants ont des équivalents.
Vibe coding	Produire vite un prototype en langage naturel, sans regarder le code ; parfait pour explorer, insuffisant pour livrer.
AI-driven development	Développement logiciel où l'IA écrit beaucoup de code, sous la direction et le contrôle d'ingénieurs.

Tronc commun : contexte, risques et coûts (rappel)

Modules 1, 3 et 6

Terme	En une phrase
Context window	Tout ce que le modèle « voit » au moment de répondre : consignes, historique, pièces jointes, résultats des tools.
Session	Une conversation ; elle accumule du contexte, utile ou parasite.
Mémoire	Informations que l'assistant réinjecte automatiquement d'une session à l'autre (profil, préférences) ; souvent désactivable.
Grounding	Ancrer la réponse dans des sources vérifiables plutôt que dans la « mémoire » du modèle.
RAG (Retrieval-Augmented Generation)	Technique de grounding : on cherche les bons extraits de documents internes et on les ajoute au contexte avant de répondre.
Cache	Mise en mémoire du début du contexte pour ne pas le repayer plein tarif à chaque message (il est relu à prix réduit) ; changer de modèle ou de tools en cours de route le perd.
Human-in-the-loop	Un humain qualifié valide la sortie de l'IA *avant* qu'elle soit utilisée.
Prompt injection	Instructions malveillantes cachées dans un contenu (email, document, page web) qui détournent un assistant.
Fuite de données	Données confidentielles envoyées à un assistant non maîtrisé (version grand public).
Plafond de dépense (spend cap)	Limite de budget configurée pour éviter les factures surprises.

Rappel de la Foundation

Termes de la Foundation repris dans les modules 1 à 6

Terme	En une phrase
Agent de code	Agent qui lit, modifie et exécute du code dans un dépôt (Copilot, Claude Code, Codex, Cursor…).
System prompt	Consignes que le harness place en tête de chaque appel, avant les instructions projet et la conversation.
Tool definitions	Nom, description et paramètres de chaque tool, envoyés au modèle à chaque appel : ils pèsent dans la context window et se lisent comme un prompt.
Instructions projet (`AGENTS.md`)	Fichier versionné dans le dépôt, chargé à chaque session : conventions, commandes, pièges. C'est l'équivalent dev des instructions personnalisées.
Demande initiale	Premier message de la session, cadré pour la tâche du jour : objectif, fichiers clés, commandes, invariants, pièges. Pour une tâche longue, on l'écrit dans un fichier rechargé à chaque session.
Invariant	Règle qui doit rester vraie quoi que fasse l'agent (un contrat d'API, une règle métier, un test qui doit passer).
Niveau d'autonomie	Ce que l'agent peut faire sans demander : proposer, modifier après accord, ou agir seul.
Permissions	Les actions autorisées à l'agent (lire, écrire, lancer une commande, accéder au réseau).
Revue IA	Relecture d'un diff par un agent ; elle aide l'humain, elle ne décide pas du merge.
Progressive disclosure	L'agent ne voit d'abord que le nom et la description d'une skill, et ne charge son contenu que quand il en a besoin.
Context rot (en business : dérive de contexte)	Dégradation des réponses quand la context window se remplit d'ajouts, de corrections et d'informations périmées ou contradictoires : la session reste crédible mais s'éloigne du besoin initial.
Compaction	Résumé automatique ou demandé de la session pour libérer de la context window.
Lost in the middle	Le modèle exploite moins bien ce qui se trouve au milieu d'un long contexte.
Trio létal	Données privées + contenu non fiable + capacité d'envoyer vers l'extérieur : la combinaison qui rend une fuite possible.

Termes propres à l'Avancé : sous le capot

Module 1

Terme	En une phrase
Prefill / decode	Les deux temps d'une réponse : lire tout le contexte, puis générer token par token.
KV cache	Mémoire de calcul du modèle qui évite de relire le contexte déjà traité.
Prompt caching	Le fournisseur réutilise le début inchangé du contexte d'un appel à l'autre, moins cher et plus rapide.
Reasoning tokens (effort)	Tokens de « réflexion » que le modèle génère avant de répondre ; ils améliorent les tâches difficiles et se paient.
SLM	Petit modèle, souvent local et spécialisé, moins cher et plus rapide qu'un LLM.

Termes propres à l'Avancé : le harness et le contexte

Modules 2 et 3

Terme	En une phrase
Agentic debt	Dette créée quand on empile sur son harness des réglages, extensions et contournements qu'on ne maîtrise plus.
Allowlist	Liste des commandes que l'agent peut lancer sans demander. Ce n'est pas une sandbox.
Sandbox	Environnement isolé qui limite ce que l'agent peut toucher, même si on l'a autorisé.
Hook	Action déterministe déclenchée par le harness à un moment précis (avant une commande, après une édition), quoi que dise le modèle.
Sous-agent (subagent)	Agent lancé par l'agent principal dans un contexte isolé, qui ne renvoie que son résultat.
Context engineering	Choisir ce qui entre dans la context window, quand et sous quelle forme.
Recherche agentique	L'agent cherche lui-même dans le code (grep, lecture de fichiers) au lieu de recevoir des extraits préparés.
Embeddings / indexation du code	Représentation du code qui permet de retrouver les passages pertinents par le sens ou par la structure.

Termes propres à l'Avancé : spec et qualité

Module 4

Terme	En une phrase
SDD (Spec-Driven Development)	Développement où la spec versionnée est la source de vérité que l'agent implémente et que les tests vérifient.
Spec-first / spec-anchored / spec-as-source	Les trois postures du SDD : la spec sert à démarrer, reste maintenue avec le code, ou devient la seule chose qu'on édite.
Spec-centric / workflow-centric	Les deux familles d'outillage du SDD : la référence est la spec versionnée, ou une discipline de travail encodée en skills.
Drift	Écart qui s'installe entre la spec et le code.
Tests de construction / tests fonctionnels	Tests qui aident l'agent à converger sur l'implémentation, jetables / tests des effets observables, durables : la spec exécutable.
Mutation testing	Technique qui modifie le code exprès pour vérifier que les tests détectent le changement.

Termes propres à l'Avancé : patterns, parallélisme et gouvernance

Modules 5 et 6, et « Au-delà du poste »

Terme	En une phrase
ADE (Agentic Development Environment)	Poste de travail qui pilote des sessions d'agent (un worktree par session, terminaux, diffs, PR) au lieu d'éditer des fichiers ; Conductor et Orca en sont des exemples.
Agent as a judge (LLM-as-a-judge)	Un agent distinct de l'auteur évalue une sortie avec une grille, dans un contexte vierge ; il prépare la preuve, l'humain décide.
Best-of-N	Même tâche lancée N fois, chaque tentative isolée ; les tests puis un juge gardent la meilleure. Coût × N.
Evals	Jeux de tests qui mesurent la qualité des réponses d'un agent ou d'une skill.
Fiabilité composée	La fiabilité d'une chaîne d'agents est le produit de celle de chaque étape : 0,95 sur 10 étapes donne environ 60 %.
Gateway (AI gateway)	Point de passage unique entre les agents et les fournisseurs de modèles : routage, coût, traçabilité, règles.
GOAP (Goal-Oriented Action Planning)	Un planificateur cherche, parmi des actions déclarées, la suite la moins coûteuse qui mène à l'état but, et replanifie si le monde change ; vient de l'IA de jeu vidéo.
Guardrails	Contrôles en entrée, en sortie et sur les actions d'un agent.

Terme	En une phrase
Orchestrateur	Logiciel ou agent qui répartit le travail entre plusieurs agents et assemble leurs résultats.
Plan-and-Execute	Le LLM écrit d'abord un plan, relu, puis l'exécute étape par étape.
Précondition / effet	Ce qui doit être vrai avant une action, et ce qu'elle rend vrai après ; la base d'un plan GOAP.
Quality gate	Contrôle automatique qu'une PR doit passer avant le merge (tests, analyse statique, evals, revue agentique).
ReAct	Boucle Pensée → Action → Observation : le modèle décide à chaque tour, les sorties des tools servent de juge.
Reflection	L'agent relit et corrige sa propre sortie ; peu cher, mais il garde ses angles morts.
Vendor lock-in	Dépendance à un fournisseur qui rend coûteux un changement d'outil ou de modèle ; formats ouverts et gateway la réduisent.
Worktree (git worktree)	Deuxième répertoire de travail rattaché au même dépôt, sur une autre branche : deux agents peuvent travailler en parallèle sans se gêner.

FICHE DE RÉFÉRENCE

Au-delà du poste (pour aller plus loin)

Ce que change l'agent pour l'entreprise, au-delà du poste du dev : guardrails, AI gateway, budget caché, réversibilité, revue à trois. Ces slides ont quitté le deck de la formation (la gouvernance est traitée côté tech managers, dans la formation « Le nouveau PDLC IA ») ; la fiche reste comme référence, à lire seul.

Où placer les contrôles

Slide retirée du deck · pour aller plus loin : « Où placer les contrôles »

Garde	Ce qu'elle contrôle
Garde d'entrée	injection, jailbreak ; données perso, secrets → masquer ; sujets interdits ; taille, débit, budget
Garde de sortie	toxicité, hors périmètre ; fuite de données ; ancrage (grounding) ; format / schéma valide
Garde d'action	liste blanche de tools ; sandbox, conteneur, réseau ; droits minimaux (tokens) ; validation humaine

> Les gardes d'entrée et de sortie **réduisent** le risque. La garde d'action le **borne** : un agent sans réseau ne peut pas exfiltrer.

- Défense en profondeur : aucun étage n'est suffisant seul.
- Les gardes d'entrée et de sortie sont souvent elles-mêmes des modèles (classifieurs, petits modèles), donc faillibles, avec de la latence et du coût. On les place sur les données externes non fiables (prompt injection) et sur les sorties qui partent vers des clients.
- La garde d'action est déterministe : un refus sur `git push` ne se contourne pas par un prompt astucieux. Pour un agent de code, c'est le premier investissement : permissions du harness, sandbox ou conteneur, tokens à portée minimale.
- **Trio létal** : données privées + contenu non fiable + capacité d'envoyer vers l'extérieur. Casser un des trois, c'est le rôle de la garde d'action.

L'AI gateway : un seul point de passage

Slide retirée du deck · pour aller plus loin : « L'AI gateway : un seul point de passage »

Un **proxy entre vos agents et les fournisseurs de modèles** : une seule API côté agents, N fournisseurs derrière, aucune clé de fournisseur sur les postes.

Fonction	Ce qu'elle apporte
Routage	choix du modèle, **fallback** si un fournisseur tombe
FinOps	quotas et **plafond de dépense** par équipe

Fonction	Ce qu'elle apporte
Observabilité	tokens, coût et latence par appel (souvent en OpenTelemetry)
Emballement	coupe l'agent coincé dans une boucle
Guardrails	masquage, détection d'injection : appliqués **à tous**
Traçabilité	modèles autorisés, résidence des données, audit

- La gouvernance sans modifier les agents : ajouter un masquage ou changer de fournisseur se fait dans la gateway, une fois.
- Le **plafond de dépense** est le garde-fou le plus simple, à poser dans la gateway.
- Attention au **cache sémantique** de certaines gateways : il renvoie une réponse déjà générée pour une question « proche ». Utile pour une FAQ, dangereux pour un agent de code. À ne pas confondre avec le **prompt caching** du fournisseur (module 1), qui ne change pas la réponse.

Le budget caché : au-delà des tokens

Slide retirée du deck · pour aller plus loin : « Le budget caché : au-delà des tokens »

Plus d'agents, c'est plus de PR. Donc plus de **builds**, de **tests**, de **mutation testing** : la facture est **double**, les tokens et les **runners CI**.

Étape	Effet
Adoption des agents	plus de travail en parallèle
Multiplication des PR	plus de CI, de tests, de mutation
Résultat	facture tokens + facture runners

> Un budget IA, c'est **tokens + CI + stockage + temps de revue**. Le temps de revue humaine est le poste le plus oublié, et c'est lui qui limite le parallélisme.

Réversibilité : éviter le vendor lock-in

Slide retirée du deck · pour aller plus loin : « Réversibilité : éviter le vendor lock-in »

Principe	Pourquoi
Harness ≠ fournisseur	l'adhérence est dans le harness ; les modèles changent tous les trimestres
Modèle via la gateway	basculer sans réécrire l'outillage
Formats ouverts	`AGENTS.md`, `SKILL.md`, MCP survivent à un changement d'outil
Résidence des données	à exiger au contrat : aucune donnée ne sert au réentraînement, hébergement connu

Modèle via la gateway : les agents appellent un nom interne (ex. `code-standard`), la gateway choisit le modèle réel derrière. Changer de modèle devient un réglage côté gateway, à condition que le harness accepte une adresse d'API personnalisée.

Choisir un harness, un modèle, un fournisseur : trois décisions désormais séparables, selon trois postures.

Posture	Exemples	Ce que ça implique
Mono-fournisseur	Claude Code	le harness impose la famille de modèles : cohérence maximale, arbitrage impossible ; l'hébergement reste négociable
Catalogue multi-éditeur	GitHub Copilot	un contrat, plusieurs modèles au choix en session : arbitrage coût / puissance sans changer d'outil
Agnostique	OpenCode, Pi, Aider ; Codex par configuration (fournisseurs compatibles API OpenAI, modèles locaux)	le fournisseur est un paramètre : liberté totale, intégration (permissions, support) à votre charge

> **Tip du module** : Tracé et portable : un agent qu'on peut **auditer**, et **quitter**. Commits tracés, specs versionnées, appels journalisés par la gateway, formats ouverts.

Pair-programming à trois

Slide retirée du deck · pour aller plus loin : « Pair-programming à trois »

Le dev et un relecteur (deux personnes) + l'agent de revue, **en même temps**.

Cycle de PR asynchrone	2 personnes + 1 agent, en synchrone
1. Le dev soumet la PR	1. L'agent de revue (CI) produit son rapport
2. Le relecteur commente (24 h d'attente)	2. Le dev et le relecteur trient les remarques en direct
3. Le dev corrige (12 h d'attente)	3. L'agent applique les corrections

> Moins d'attente, des retours plus pertinents, une validation immédiate. Et un moment où la connaissance circule.

À retenir

1. La **garde d'action** borne le risque, les gardes d'entrée et de sortie le réduisent.
2. La **gateway** route, plafonne, observe et trace, pour toute l'entreprise.
3. Un budget IA compte aussi la **CI** et le **temps de revue**.
4. Un agent tracé et portable : on peut l'**auditer**, et le **quitter**.

Mots-clés : guardrails, gateway, prompt injection, trio létal, plafond de dépense, vendor lock-in, human-in-the-loop — voir le glossaire.